DIGITAL TWIN APROACH TO DESIGN AND DEVELOPMENT OF GIMBAL

TOBB UNIVERSITY OF ECONOMICS AND TECHNOLOGY

MECHANICAL ENGINERING
SENIOR DESIGN PROJECT REPORT

Hilmi Fatih Er	**181501039**
Okan Berhoğlu	**181501061**
Ömer Dehan Özboz	**211501067**
Tarıksafa Soylu	**181501050**

24/04/2023

ACKNOWLEDGEMENT

We would like to thank our supervisor Associate Professor Hakkı Özgür Ünver for his guidance, support, and considerateness. It was a great honor to work with him and our cooperation influenced our work.
We would like to thank our supporting company TEKNOPAR and company supervisor Muhammet Furkan Işık for his guidance, support, and considerateness.
Also, we would like to express our immense gratitude to Associate Professor Recep Muhammet Görgülüaslan, and Bahman Payegozar for their assistance on 3D printed parts production and design with their vast knowledge and experience.

DIGITAL TWIN APROACH TO DESIGN AND DEVELOPMENT OF GIMBAL

ABSTRACT

Gimbal systems are used in various engineering applications such as military systems, industrial recording systems and telescope applications. Their configurations are designed according to the types of application and desired performance requirements. The essential aim of these systems is to compensate the disturbance effects in order to stabilize inertially stabilized platform and positioning to the desired point.

Digital twin technology is the creation of a one-to-one copy of any object, device, or service in the digital environment, even without physically performing it. With smart sensors or Internet of Things (IoT) sensors, full-time data from the physical environment is transferred as input to the digital twin.

In this project, first, a detailed kinematic mathematical model of a 3-DOF gimbal system containing rotation matrices and frames were obtained. And the system was simulated in MATLAB software and web server separately with using sensor measurements.

To create the gimbal mechanism, links and the base that carry whole system are designed in SOLIDWORKS environment to be printed in 3D printer. With production of these parts and bought hardware**,** in the final part of this study, the gimbal system was assembled, and microcontroller was programmed with kinematic model algorithm. After that performance has been inspected and tested to work properly.

As soon as these operations were made, virtual (web server) and real environment connection and communication were generated in accordance with the purpose of digital twin approach. And simultaneous movement and communication was provided finally.

SANAL İKİZ YAKLAŞIMI İLE GİMBAL TASARIMI VE GELİŞTİRİLMESİ

ÖZET

Gimbal sistemleri, askeri sistem, endüstriyel kayıt sistemleri ve teleskop uygulamaları gibi çeşitli mühendislik uygulamalarında kullanılmaktadır. Uyarlamalar, uygulama tiplerine göre ve istenen performans gereksinimlerine göre tasarlanır. Bu sistemlerin temel amacı ataletsel olarak stabilize edilmiş platformu bozulma etkilerinden korumak ve konumlandırmayı istenen noktaya stabilize etmektir.

Dijital ikiz teknolojisi, herhangi bir nesnenin, cihazın veya hizmetin fiziksel olarak gerçekleştirmeden dahi birebir kopyasının dijital ortamda oluşturulmasıdır. Akıllı sensörler veya Nesnelerin İnterneti (IoT) sensörleri ile fiziksel ortamdan tam zamanlı veriler girdi olarak dijital ikize aktarılır eş zamanlı simülasyon sağlanır.

Bu projede, ilk olarak, dönme matrisleri ve çerçeveleri içeren 3 serbestlik dereceli bir yalpalama sisteminin ayrıntılı bir kinematik matematiksel modeli elde edilmiştir. Ve sistem sensör ölçümleri kullanılarak Matlab yazılımında ve web sunucusunda ayrı ayrı simüle edilmiştir.

İkinci adım olarak Gimbal mekanizmasını oluşturmak için tüm sistemi taşıyan bağlantılar ve taban 3D yazıcıda basılmak üzere SOLIDWORKS ortamında tasarlanmıştır. Bu parçaların üretimi ve satın alınan donanım ile çalışmanın son aşamasında gimbal sistemi kurulmuş ve kinematik model algoritması ile mikrodenetleyici programlanmıştır. Daha sonra düzgün çalışmasını doğrulamak amacıyla performans incelenmiştir ve test edilmiştir.

Bu işlemler yapılır yapılmaz dijital ikiz yaklaşımının amacına uygun olarak sanal ve gerçek ortam bağlantısı ve haberleşmesi oluşturulmuştur. Ve eş zamanlı hareket ve iletişim sağlanıştır.

CONTENT

Page

LIST OF TABLES

Page

LIST OF FIGURES

Page

1. INTRODUCTION

1.1 Background

Inertial stabilization platforms (ISPs) are used in lots of various engineering applications such as weapon systems, telescopes, and cameras. Their configurations are designed according to the types of application and desired performance requirements. The essential aim of these systems is to compensate the maneuvers, platform motions, vibrations or hold steady, a pointing vector, namely line-of-sight (LOS), in an inertial space by means of electro-optic sensors such as inertial measurement units (IMUs), encoders and cameras. The disturbances cause decreasing the pointing accuracy of the ISPs. By utilizing Gimbal Systems (ISP Systems) a desired configuration on inertial frame can be obtained [1].

Digital twins are models of specific real-world entities equipped with sensors that constantly update their virtual counterparts with granular, high-quality data in real time, as opposed to simulations that run in completely virtual environments separate from the outside world. Businesses and organizations use digital twins to design, build, operate and track products throughout their lifecycle. Armed with up-to-date data on physical objects, digital twins use artificial intelligence and machine learning to build detailed predictive models and predict more accurate results than most simulations.

In this project a 3 Axis Gimbal System will be designed. The project consists of four main parts which are modeling, simulation, assembly, and synchronization of the final products.

The system which will be developed has a base for carrying the system, links for 3-Axis stabilization, 3 BLDC motors with integrated encoder and an IP camera, and an IMU sensor in strapdown configuration for estimation of the base in inertial frame.

When the gimbal is yet to be constructed, simulation studies gain more importance for the system designers. There are lots of applications and software to simulate kinematic characteristics of robot systems. Mathematical modeling of the gimbal takes the most importance for this study because the system is still in preliminary design phase. With preliminary construction of a detailed system model, designers can observe the

performances of the system even if the physical system is not present. Moreover, simulation studies are faster, desired data can be acquired easily and system scenarios can be simulated any number of times.

1.1 Literature Survey

In literature, gimbal systems have been investigated with various different features. The differences can be categorized as; physical features, control technique features, sensor availability and placement related features, and rigid body motion formulation techniques.

Physical features can be subdivided considering the degree of freedom (DOF) of the system. If an exact configuration at the final link of the system (Camera Orientation) is desired, at least 3-DOF freedom is needed. However, with orthogonally placed 3-DOF rotation axes, a Gimbal Lock problem may occur. Gimbal lock can be defined as the loss of a DOF at a singularity point of the system. On Figure 1.1, gimbal lock problem is represented. As shown on Figure 1.1, if the joint represented as "2" rotates to the point where the rotation axis of joints "1" and "3" coincide, a DOF is lost. One way to eliminate this phenomenon is to have a redundant fourth joint. However, the fourth joint results in a more expensive system due to the fourth motor, also other motor selections should be revisited since more torque requirements may be needed [2].

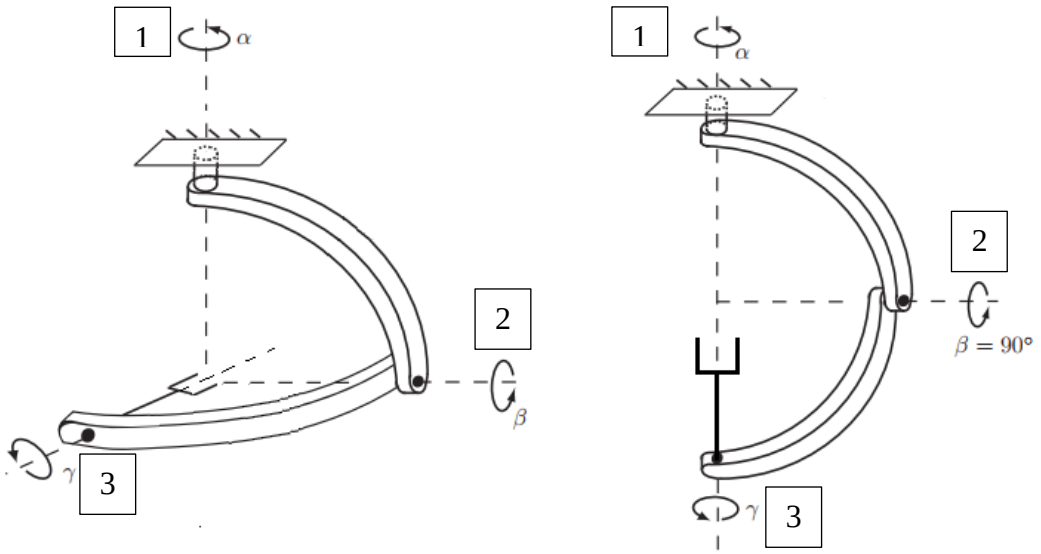


Figure 1.1: Gimbal lock representation [3].

However, it should be noted that, if operating zone of axis “2” can be defined as to be “$-90^{o} < \beta < 90^{o}$”, the gimbal lock can be prevented. Another way to prevent gimbal lock of the system is to have non-orthogonal axis selection, in that case axis “2” cannot cause the coincidence of first and third rotation axes.

Another DOF selection can be 2-DOF. In camera systems, the desired line of sight can be achieved with 2-DOF, and in computer environment third degree of rotation (image rotation) can be achieved.

With 3-DOF systems, exact orientation of the final link (Camera Orientation) can be achieved without the need of computer image rotation. In this project, 3-DOF with orthogonal axes gimbal system will be analyzed and designed.

For inertial measurement unit (IMU) sensor, there are mainly two placement selections. These are called direct stabilization and indirect stabilization. In direct stabilization, the IMU is attached to the final link of the stabilization system, such as on the camera. On indirect stabilization, the IMU is attached to the base of the system where the gimbal system is connected. This attachment of the IMU to the base is named strapdown configuration [4].

Another design selection of the systems is related to control input to the axis motors. In literature, there are 2 main control input selections, these are torque input to the system where DC motor’s torque-current relation is defined, and control is achieved. One other way is the usage of angular velocity as the control input; in this case by using required angle values of each axis and encoder readings from motors, control system can be designed.

With control input chosen as torque input, the dynamical model of the system should be represented, where the mass imbalances and moment of inertia of links with respect to rotation axes should be formulated according to the motor angle in every configuration. In that case, every motor’s angle with respect to the zero instance (starting instance) affects the center of mass of the system. In literature, linearization around selected equilibrium points have been utilized [5].

In the case velocity input chosen as control input, main assumption is that the motors are able to provide adequate torque for the demanded angular velocity. However, in that case, dynamic model of the system is not required [3].

3D Rigid Body Motion can be defined as representing a rigid body in 3D Space, and there are many representations utilized in the literature. Euler Angles, quaternion,

product of exponential, direct cosine matrices (DCM), and Roll-Pitch-Yaw are the main formulations seen in the literature.

After modeling was completed, simulation codes were started to simulate system before creating real working system environment. The main aim of building a virtual twin is to realize system and provide simultaneous data transfer between virtual and real model. For assembly of system feasible and more useful design models were searched and compared to create.

2. STATE OF THE ART

Gimbal technology has come a long way in recent years with significant advances in stabilization, control systems and features. State-of-the-art gimbal technology includes the use of brushless motors that are more efficient, reliable and precise than their brushed counterparts. In addition, the development of advanced control algorithms and sensors such as gyroscopes and accelerometers has allowed for more precise and accurate stabilization of cameras and other equipment. Many modern gimbals also incorporate advanced features such as object tracking, time-lapse modes, and wireless connectivity, making them an essential tool for videographers and photographers alike. With continued advances in technology, gimbals are likely to become even more versatile and useful in the years to come.

Recently Inertially Stabilized Platform Technology has been used on many applications such as;

- Telescope Applications
- Military Observational System (UAV, UGV, etc.), and Locking on the Target
- Industrial Gimbal Systems for Professional Recording
- Personal Gimbal Systems

In this project, an observational system for an Unmanned Ground Vehicle is being designed and produced which transfers synchronous data between computer and Gimbal System.

Figure 2.1: ISP usage examples [6]-[7]-[8].

3. GOALS OF THE PROJECT

3.1 Goals

- To produce a gimbal for industrial usage
- To implement different control methods for stabilization
- To implement and develop Digital Twin of Gimbal
- To transfer data wirelessly between gimbal and digital twin for real time synchronization

3.2 Motivations

- Importance and desire for implementation of dissociating the required motion of the platform from the chaotic motion of the vehicle for various applications.
- Rise in the need and market share of gimbal system in both personal and industrial usage.
- Recent increase in the implementation of Digital Twin Methodology on various engineering applications.

4. PRODUCT DEVELOPMENT PROCESS

The product development process for a gimbal and digital twin involves several stages. The first stage involves researching the market and surveying literature. During this stage, the group might conduct surveys for hardware, system mechanics, electronic and modelling. The next stage is concept development and evaluation, during which the group generates ideas for features, design, and functionality. Once a concept is chosen and alternatives are evaluated, the design phase begins. The design team creates detailed drawings, specifications and 3D parts, modelling team finds and gets model and simulate it and hardware team provides the tools and configurate them. After assembling and software part begins. With so much testing, the gimbal is subjected to various scenarios and environments to identify any issues or areas for improvement. Once the prototype is refined and working, the group moves to the improve and quality phase. After manufacturing, the team conducts requirement checks and tests to ensure that the product meets all wanted criteria. Finally, the gimbal and digital twin is created.

5. CONCEPT ALTERNATIVES AND EVALUATION

Table 5.1: Concept alternatives.

1 5	Brushed DC Motor	Brushless DC Motor	3D Printed Material	Conventional Production Methods	2-DOF (Additional Graphical DOF needed)	3-DOF	4-DOF
Low Cost	4	2	3	4	4	3	1
Weight	5	5	5	4	5	4	3
Ease of Application	2	5	5	3	1	3	1
Usability	3	5	NA	NA	4	4	2
Built-In Sensors	1	5	NA	NA	NA	NA	NA
Precision	3	5	NA	NA	NA	NA	NA
Design Flexibility	NA	NA	5	2	NA	NA	NA
Reliability in Every Configuration (Avoiding Gimbal Lock)	NA	NA	NA	NA	1	1	5
	18	**27**	**18**	13	15	**15**	12

In this project cost, weight, ease of application, usability etc. variables were evaluated for choosing specific tools, production method and concept of gimbal.

Firstly, BLDC motors were chosen because of ease, built-in sensor advantage, higher precision compared to Brushed DC motor and servo motor.

Secondly, links and base were supposed to be produced. 3D printing-creating method were much more feasible. Design flexibility was the main decisive factor. Because the gimbal project was delicate, precise and relatively small in size. More changes and fine design could be made with 3D printer.

Finally, there were three different concepts of gimbal that had different degrees of freedom. In total calculation of all variables' gains and adding more source advantage 3-DOF were the choice.

6. PRODUCT ARCHITECTURE

Product architecture plays a critical role in determining how a product's components are positioned and how they interact with each other. This fundamental concept includes many factors that affect the success of products and their adaptation to the market. Therefore, product architecture is seen as an indispensable tool for companies to turn their value propositions into reality. Product architecture, which is of great importance in terms of meeting customer expectations, is accepted as the cornerstone of a successful product. It covers critical components such as component optimization, usability, and modularity, which are important in creating effective product architecture.

Companies primarily rely on product architecture to gain advantage. To achieve this, engineers and designers must carefully plan the optimization of components to increase the performance and functionality of the product. In this process, factors such as material and production costs, energy efficiency and sustainability help companies reduce costs and attract consumers by developing environmentally friendly products.

Usability directly affects the user experience, and the ease with which users interact with the product and successfully achieve the purpose of the product increases customer satisfaction and continuity. For this reason, the design should be intuitive, ergonomic, and user-friendly.

Another aspect, modularity, allows the creation of products with independent and interchangeable components, providing consumers with a variety of options to meet their different needs and preferences. This simplifies the maintenance and repair of the product.

The gimbal keeps stable a device and prevents vibrations. In this way makes it possible to obtain more stable images and more professional images while shooting video can be achieved with the camera.

Gimbal design consists of mechanical, electronic and software components. The electronics part contains the circuits and motors that control the gimbal. The software part contains the software that controls the operation of the gimbal. The mechanical part includes the fasteners, motors, body of the device and other mechanical parts.

In the design, consideration should be given to factors such as device size, weight and power supply, stability, and ease of use. It is very important to choose the best possible materials, reduce the weight of the device, increase the ease of use, and give importance to ergonomics in the design. The high precision and power of the gimbal's motors will increase the stability of the device. Factors such as production method and material selection are important for the success of the product.

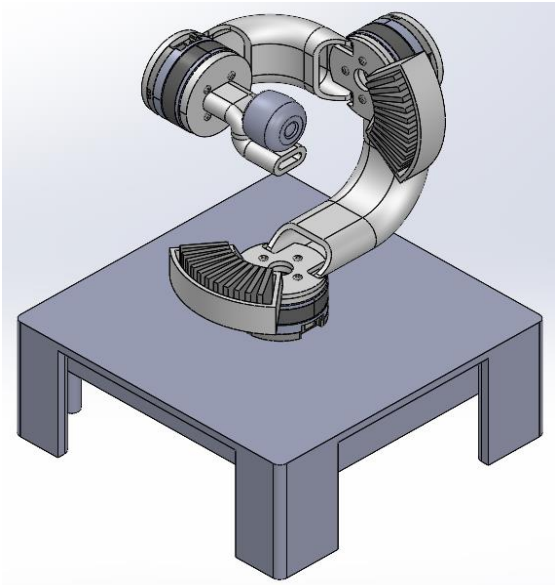


Figure 6.1: Final Design

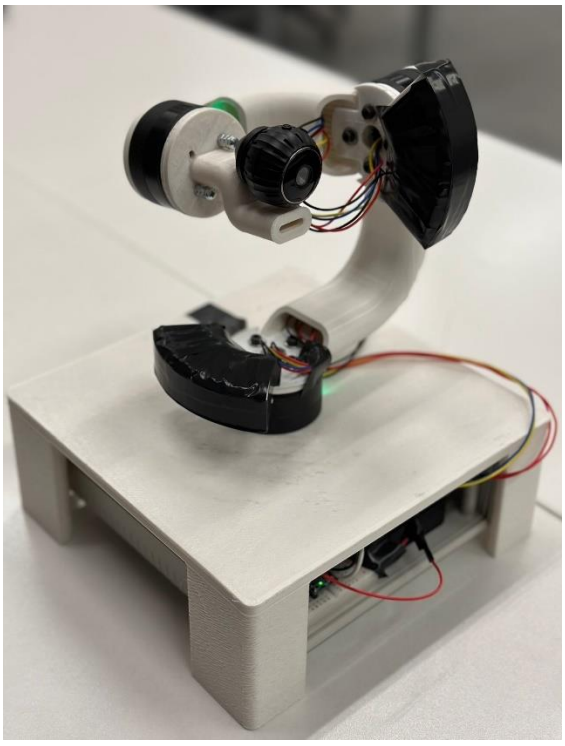


Figure 6.2: Final manufactured product.

7. ENGINEERING ANALYSIS

7.1 Mathematical Model of the Gimbal System

In this project, a 3 DOF Gimbal System will be designed. Since the orientation of these 3 degrees of freedom are located in 3D space, rigid body motion of each rotating joint should be defined. Additionally, the base where the Gimbal is attached, also move within the space, this motion also should be investigated. Since rigid body motion will be used throughout the report, a summary of the rigid body motion formulation will be provided first.

7.1.1 Rotation Matrices

In Figure 7.1, the formulation of rotation in X-Y Plane has been shown. In the figure, the point P_j is rotated by θ degrees around Z-Axis, and a new point is obtained which is called P_j^*. The x, and y coordinates of the new point can be defined as given on the equation 7.1 with respect to the angles θ, α, and the distance of the point to the origin which is denoted as , l. The result of the equation 7.1 in matrix form can be seen on equation 7.2.

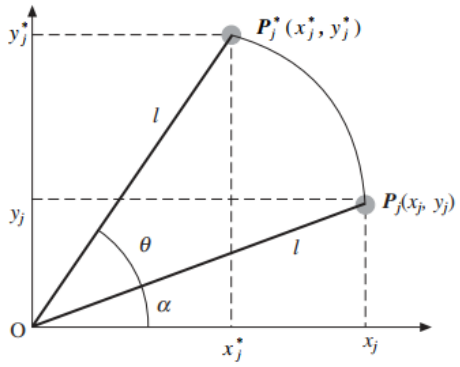


Figure 7.1: Rotation in X-Y Plane. [9]

$$x_j^* = l\,cos(\theta+\alpha) = l\,cos(\alpha)\,cos(\theta) - l\,sin(\alpha)\,sin(\theta)$$

$$\mathbf{x_j}^* = \boldsymbol{x_j\,cos(\theta) - y_j\,sin(\theta)}$$

$$y_j^* = l\,sin(\theta+\alpha) = l\,cos(\alpha)\,sin(\theta) + l\,sin(\alpha)\,cos(\theta) \tag{7.1}$$

$$\mathbf{y_j}^* = \boldsymbol{x_j\,sin(\theta) + y_j\,cos(\theta)}$$

$$\begin{bmatrix} x_j^* \\ y_j^* \end{bmatrix} = \begin{bmatrix} cos\,(\theta) & -sin\,(\theta) \\ sin\,(\theta) & cos\,(\theta) \end{bmatrix} \begin{bmatrix} x_j \\ y_j \end{bmatrix} \tag{7.2}$$

Obtained matrix form is in 2D, however can be extended into 3D form as given on the equation 7.3 below. As pointed out, the resulting rotation takes place on X-Y plane around Z-Axis. However similar formulations can be made for X-Axis, and Y-Axis rotations. On equation 7.4 and equation 7.5, these Y-Axis and X-Axis rotation formulations can be seen respectively.

$$\begin{bmatrix} x_j^* \\ y_j^* \\ z_j^* \end{bmatrix} = \begin{bmatrix} \boldsymbol{cos(\theta)} & \boldsymbol{-sin(\theta)} & \mathbf{0} \\ \boldsymbol{sin(\theta)} & \boldsymbol{cos(\theta)} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{1} \end{bmatrix} \begin{bmatrix} x_j \\ y_j \\ z_j \end{bmatrix} \tag{7.3}$$

$$P_j^* = \boldsymbol{Rot(z, \theta)} \cdot P_j$$

$$\begin{bmatrix} x_j^* \\ y_j^* \\ z_j^* \end{bmatrix} = \begin{bmatrix} \boldsymbol{cos(\theta)} & \mathbf{0} & \boldsymbol{sin(\theta)} \\ \mathbf{0} & \mathbf{1} & \mathbf{0} \\ \boldsymbol{-sin(\theta)} & \mathbf{0} & \boldsymbol{cos(\theta)} \end{bmatrix} \begin{bmatrix} x_j \\ y_j \\ z_j \end{bmatrix} \tag{7.4}$$

$$P_j^* = \boldsymbol{Rot(y, \theta)} \cdot P_j$$

$$\begin{bmatrix} x_j^* \\ y_j^* \\ z_j^* \end{bmatrix} = \begin{bmatrix} \mathbf{1} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \boldsymbol{cos(\theta)} & \boldsymbol{-sin(\theta)} \\ \mathbf{0} & \boldsymbol{sin(\theta)} & \boldsymbol{cos(\theta)} \end{bmatrix} \begin{bmatrix} x_j \\ y_j \\ z_j \end{bmatrix} \tag{7.5}$$

$$P_j^* = \boldsymbol{Rot(x, \theta)} \cdot P_j$$

These rotation matrices are part of a group of rotation matrices called special orthogonal group *SO(3)*. There are some useful properties of SO(3) matrices. One of which is given on the equation 7.6. According to equation 7.6, matrix multiplication of transpose of the rotation matrix with itself results in an identity matrix. Which suggests that, inverse of the rotation matrix equals to the transpose of the rotation matrix [3].

$$R^T R = I$$
$$\mathrm{R^T = R^{-1}} \tag{7.6}$$

This property is important, because computationally, taking transpose of a matrix is easier than taking inverse of the matrix.

In the formulations given above, the rotation processes have been defined, however there are several usages of rotation matrices. Which are given below [3].

- To represent an orientation,
- To change the reference frame in which a vector or a frame is represented,
- To rotate a vector or a frame.

The provided usages are utilized throughout the project where necessary.

With obtained rotation matrices, 3D orientation of an object can be defined by successive multiplications of these matrices. Some of which can be defined as Euler Angles. There are several different types of Euler Angles, which are defined with respect to the order of their multiplication sequence, such as ZYX Euler Angles, ZYZ Euler Angles or XYZ Euler Angles. Also, it should be noted that there are other rotation matrix representations which are not restricted to the X, Y, and Z-Axis, such as Product of Exponential Method with the usage of Rodrigues' formula [3]. Additional to these representations, unit quaternion representation is used for representing rotation.

One other representation of 3D orientation is called Yaw-Pitch-Roll (YPR) representation. The formulation of YPR representation and the ZYX Euler angles result in the same formulation.

7.1.2 Representing Base Frame

In order to represent the total orientation of the gimbal, first base motion should be understood. Figure 7.2 is an example product retrieved from Waveshare. On the figure a representative axis has been shown.

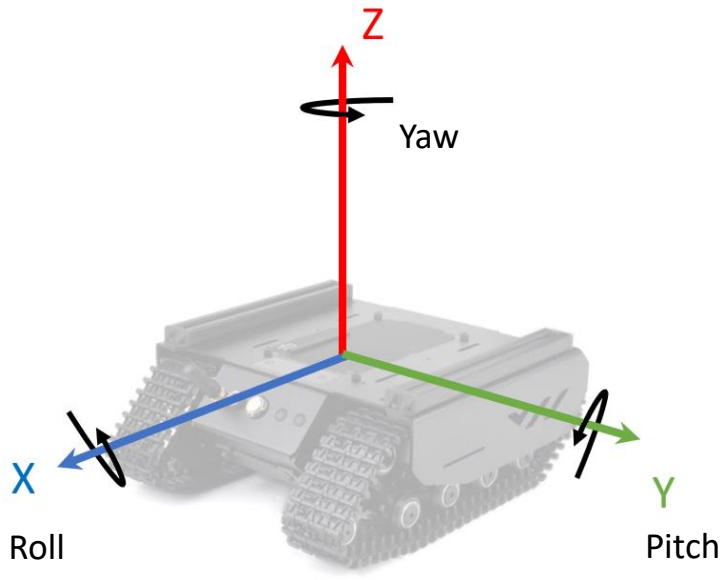


Figure 7.2: Base axes [10].

In order to represent the orientation of the Base Frame, first a reference frame called Earth Frame should be defined. Earth Frame is defined as a fixed frame in inertial space, which does not move with respect to any movement of the system. Then with successive rotations around Z, Y and X axes Base Frame is obtained [4-5-11-12-13].

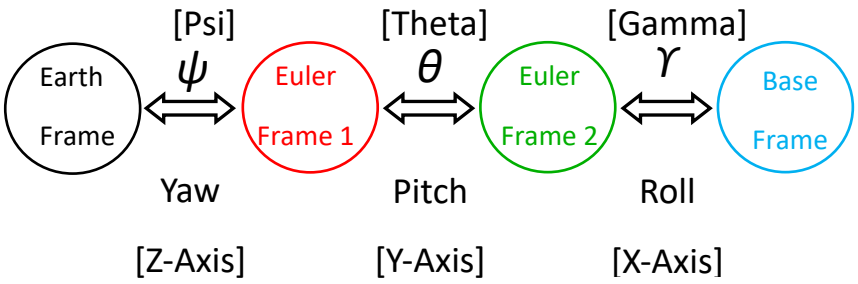


Figure 7.3: Reference frames from Earth Frame to Base Frame.

On the Figure 7.4, these successive rotations are 3D represented with their axes.

- X_e, Y_e, Z_e: Axes located in *Earth Frame*, with color Black
- X_1, Y_1, Z_1: Axes located in *Euler Frame 1*, with color Red
- X_2, Y_2, Z_2: Axes located in *Euler Frame 2*, with color Green
- X_b, Y_b, Z_b: Axes located in *Base Frame*, with color Blue

For clarity, it can be summarized as

- *Euler Frame 1* is represented as rotated with respect to *Earth Frame* around Z_e-Axis.
- *Euler Frame 2* is represented as rotated with respect to *Euler Frame 1* around Y_1-Axis.
- *Base Frame* is represented as rotated with respect to *Euler Frame 2* around X_2-Axis.

These successive rotations will be called ZYX Euler Angles. Also, on the Figure 7.4 dashed axes represent that, the axis coincide with previous frame's axis.

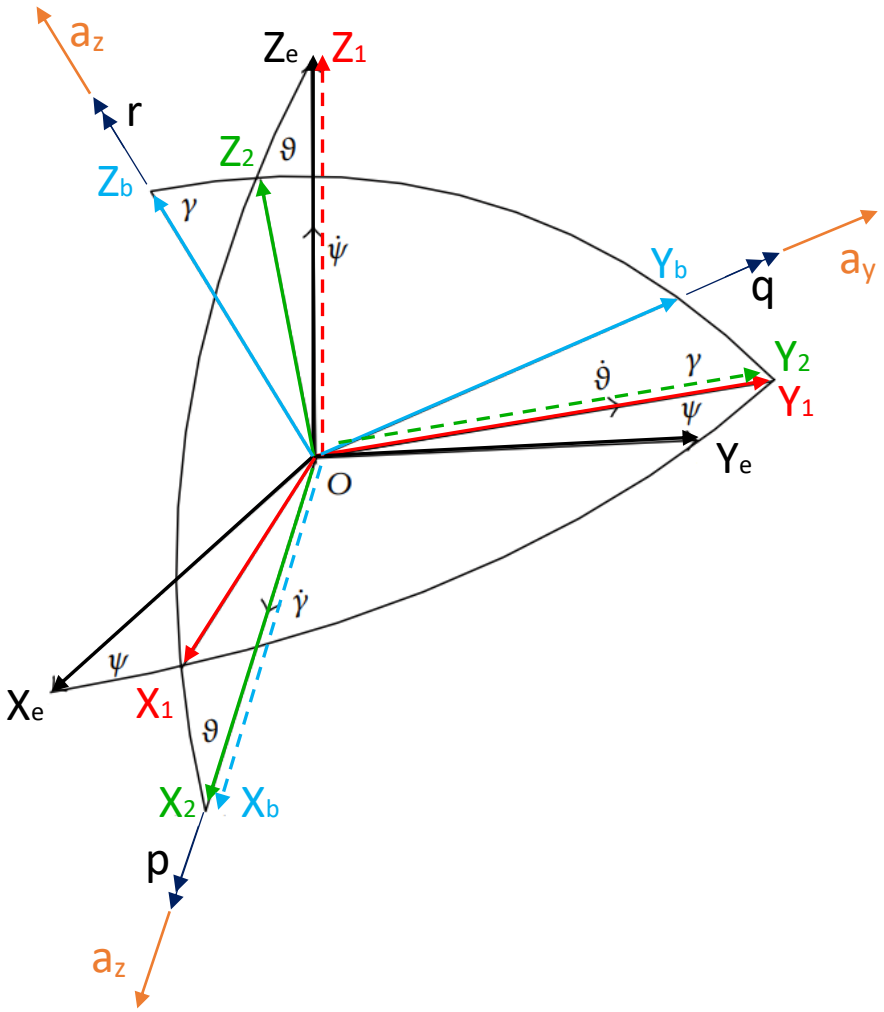


Figure 7.4: Sequence of rotations determining the base position. [13]

With defined rotation axes, and formulated equations 7.3, 7.4, and 7.5 final orientation of the base can be defined.

$$R_{e-1}(Z,\psi) = \begin{bmatrix} cos(\psi) & -sin(\psi) & 0 \\ sin(\psi) & cos(\psi) & 0 \\ 0 & 0 & 1 \end{bmatrix} \tag{7.7}$$

$$R_{1-2}(Y,\vartheta) = \begin{bmatrix} cos(\vartheta) & 0 & sin(\vartheta) \\ 0 & 1 & 0 \\ -sin(\vartheta) & 0 & cos(\vartheta) \end{bmatrix} \tag{7.8}$$

$$R_{2-b}(X,\gamma) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & cos(\gamma) & -sin(\gamma) \\ 0 & sin(\gamma) & cos(\gamma) \end{bmatrix} \tag{7.9}$$

$$\boldsymbol{R_{e-b}(\psi,\vartheta,\gamma) = R_{e-1}(Z,\psi) * R_{1-2}(Y,\vartheta) * R_{2-b}(X,\gamma)} \tag{7.10}$$

With matrix multiplication, equation 7.10 can be defined explicitly as given on the equation 7.11. Equation 7.11 represents the rotation matrix to project a vector from base frame to earth frame.

$$R_{e-b}(\psi,\vartheta,\gamma)=\begin{bmatrix} c(\vartheta)c(\psi) & s(\gamma)s(\vartheta)c(\psi)-c(\gamma)s(\psi) & c(\gamma)s(\vartheta)c(\psi)+s(\gamma)s(\psi) \\ c(\vartheta)s(\psi) & s(\gamma)s(\vartheta)s(\psi)+c(\gamma)c(\psi) & c(\gamma)s(\vartheta)s(\psi)-s(\gamma)c(\psi) \\ -s(\vartheta) & s(\gamma)c(\vartheta) & c(\gamma)c(\vartheta) \end{bmatrix} \quad (7.11)$$

7.1.3 Representing Platform Frame

On the previous section, Base Frame orientation with respect to the Earth Frame was defined. From Base Frame to Platform Frame, similar rotations can be applied. On the Figure 7.5, the final gimbal design with axes of arm links defined, can be seen at starting instance.

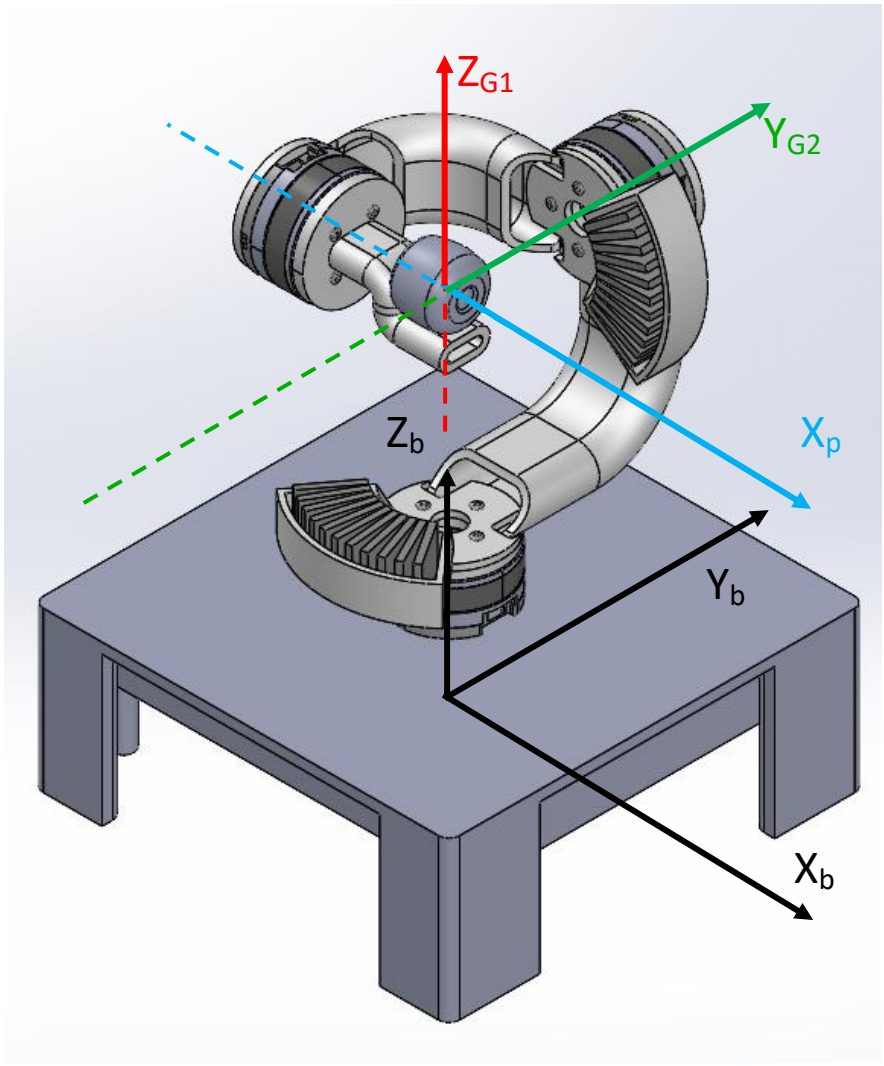


Figure 7.5: Zero-instance axes of Arm Links.

On Figure 7.6, the rotation sequence from Base Frame to Platform Frame is defined where the Platform Frame is the final link that holds the camera.

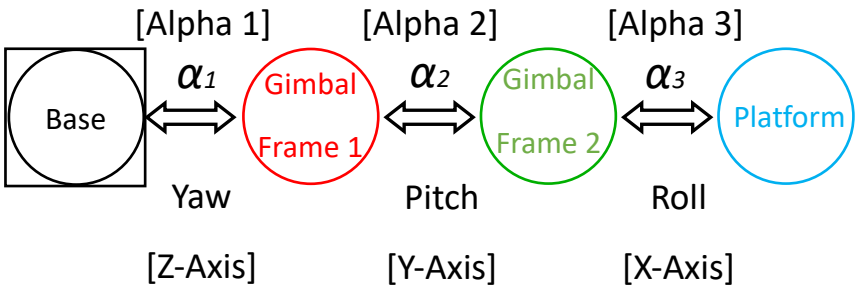


Figure 7.6: Reference frames from Base Frame to Platform Frame.

On Figure 7.5, the starting orientation of the Gimbal was shown. At the starting orientation, the axes of each frame; Gimbal Frame 1, Gimbal Frame 2, and Platform Frame coincide with each other. In order to show simplified axes representation only basic axes have been shown. On Figure 7.7 the detailed axes of frame with rotation angles can be seen.

On equation 7.12, 7.13 and 7.14, the rotation matrices can be seen.

$$R_{\mathrm{b-G1}}(\mathrm{Z},\alpha_1)=\begin{bmatrix} cos(\alpha_1) & -sin(\alpha_1) & 0 \\ sin(\alpha_1) & cos(\alpha_1) & 0 \\ 0 & 0 & 1 \end{bmatrix} \tag{7.12}$$

$$R_{G1-G2}(Y,\alpha_2)=\begin{bmatrix} cos(\alpha_2) & 0 & sin(\alpha_2) \\ 0 & 1 & 0 \\ -sin(\alpha_2) & 0 & cos(\alpha_2) \end{bmatrix} \tag{7.13}$$

$$R_{G2-p}(X,\alpha_3)=\begin{bmatrix} 1 & 0 & 0 \\ 0 & cos(\alpha_3) & -sin(\alpha_3) \\ 0 & sin(\alpha_3) & cos(\alpha_3) \end{bmatrix} \tag{7.14}$$

$$\boldsymbol{R_{b-p}(\alpha_1,\alpha_2,\alpha_3)=R_{b-G1}(Z,\alpha_1)*R_{G1-G2}(Y,\alpha_2)*R_{G2-p}(X,\alpha_3)} \tag{7.15}$$

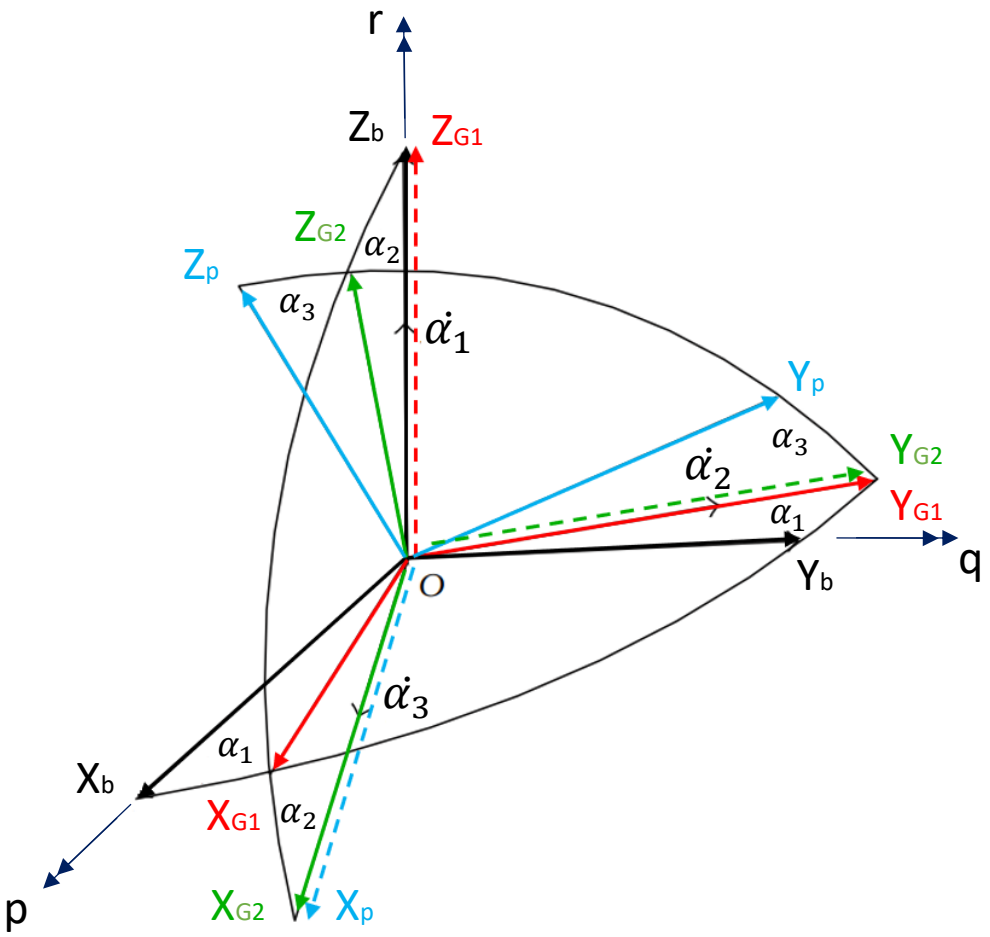


Figure 7.7: Sequence of rotations determining the platform position. [13]

On the Figure 7.7, these successive rotations are 3D represented with their axes.

- X_b, Y_b, Z_b: Axes located in *Base Frame*, with color Black
- X_{G1}, Y_{G1}, Z_{G1}: Axes located in *Gimbal Frame 1*, with color Red
- X_{G2}, Y_{G2}, Z_{G2}: Axes located in *Gimbal Frame 2*, with color Green
- X_p, Y_p, Z_p: Axes located in *Platform Frame*, with color Blue

For clarity, it can be summarized as

- *Gimbal Frame 1* is represented as rotated with respect to *Base Frame* around Z_b-Axis.
- *Gimbal Frame 2* is represented as rotated with respect to *Gimbal Frame 1* around Y_{G1}-Axis.
- *Platform Frame* is represented as rotated with respect to *Gimbal Frame 2* around X_{G2}-Axis.

With these defined rotations, using equation 7.10 and 7.*15 (Earth Frame to Base Frame, and Base Frame to Platform Frame)*, platform orientation with respect to Earth Frame can be defined as given on the equation 7.16.

$$\boldsymbol{R_{e-p}(\psi, \vartheta, \gamma, \alpha_1, \alpha_2, \alpha_3)} = \\ \boldsymbol{R_{e-1}(Z, \psi) * R_{1-2}(Y, \vartheta) * R_{2-b}(X, \gamma) * R_{b-G1}(Z, \alpha_1) * R_{G1-G2}(Y, \alpha_2) * R_{G2-p}(X, \alpha_3)} \tag{7.16}$$

7.1.4 IMU Sensor

For estimating the Base orientation, strapdown configuration for inertial measurement unit (IMU) is used. Which means that, IMU is located on the Base Frame. The IMU unit is InvenSense MPU 6050, which has 3-Axis Accelerometer and 3-Axis Gyro. With these data, the base orientation can be defined. On Figure 7.4, [u, v, w] vectors represent the acceleration vectors on the Base Frame -measured with IMU Accelerometer-, and [p, q, r] vectors represent angular velocity of the Base Frame -measured with IMU Gyro-.

- Base Position with respect to Earth Frame: $[x, y, z]^T$
- ZYX Euler Angles: $[\psi, \theta, \gamma]^T$
- Linear Accelerations on Base Frame: $[u, v, w]^T$
- Angular Velocities on the Base Frame: $[p, q, r]^T$

7.1.4.1 Estimating Base Euler Angles (ψ, θ, γ)

Base Euler Angle Representation with Gyro Data:

The obtained angular velocity measurements are on the Base Frame, however Euler Angles are located on different frames, therefore for estimation of Euler Angles, some rotations should be carried out. If rate of change of the Euler Angles ($\dot{\psi}$, $\dot{\theta}$, $\dot{\gamma}$) are defined on their respective frames as shown on the Figure 7.4, with the help of rotation matrices, these rates can be represented on the Base Frame. On equation 7.17 this formulation can be seen. Since $\dot{\gamma}$ coincide with the Base Frame, it is used as it is. However, in order to project $\dot{\theta}$ to Base Frame, a rotation matrix of "$R_{b-2}(X, \gamma)$" should be used. Also for projecting $\dot{\gamma}$ successive rotations of "$R_{b-2}(X, \gamma)$" and "$R_{2-1}(Y, \vartheta)$" [14].

$$\begin{bmatrix} p \\ q \\ r \end{bmatrix} = \begin{bmatrix} \dot{\gamma} \\ 0 \\ 0 \end{bmatrix} + R_{b-2}(X, \gamma) \begin{bmatrix} 0 \\ \dot{\vartheta} \\ 0 \end{bmatrix} + R_{b-2}(X, \gamma) * R_{2-1}(Y, \vartheta) \begin{bmatrix} 0 \\ 0 \\ \dot{\psi} \end{bmatrix} \tag{7.17}$$

If equation 7.17 is organized, equation 7.18 can be obtained. Which relates the rate of changes of the Euler Angles to IMU angular velocities.

$$\begin{bmatrix} p \\ q \\ r \end{bmatrix} = \begin{bmatrix} 1 & 0 & -sin(\theta) \\ 0 & cos(\gamma) & cos(\theta) * sin(\gamma) \\ 0 & -sin(\gamma) & cos(\theta) * cos(\gamma) \end{bmatrix} \begin{bmatrix} \dot{\gamma} \\ \dot{\vartheta} \\ \dot{\psi} \end{bmatrix}$$

$$\begin{bmatrix} p \\ q \\ r \end{bmatrix} = T^{-1}(\psi, \vartheta, \gamma) \begin{bmatrix} \dot{\gamma} \\ \dot{\vartheta} \\ \dot{\psi} \end{bmatrix} \tag{7.18}$$

Equation 7.18 can be rearranged, as a result equation 7.19 can be obtained. Then by integrating the equation 7.19, ZYX Euler Angles can be obtained.

$$\begin{bmatrix} \dot{\boldsymbol{\gamma}} \\ \dot{\boldsymbol{\vartheta}} \\ \dot{\boldsymbol{\psi}} \end{bmatrix} = \begin{bmatrix} \mathbf{1} & \boldsymbol{sin(\gamma) * tan(\theta)} & \boldsymbol{cos(\gamma) * tan(\theta)} \\ \mathbf{0} & \boldsymbol{cos(\gamma)} & \boldsymbol{-sin(\gamma)} \\ \mathbf{0} & \boldsymbol{sin(\gamma) * sec(\theta)} & \boldsymbol{cos(\gamma) * sec(\theta)} \end{bmatrix} \begin{bmatrix} \boldsymbol{p} \\ \boldsymbol{q} \\ \boldsymbol{r} \end{bmatrix}$$

$$\begin{bmatrix} \dot{\boldsymbol{\gamma}} \\ \dot{\boldsymbol{\vartheta}} \\ \dot{\boldsymbol{\psi}} \end{bmatrix} = \boldsymbol{T(\psi, \vartheta, \gamma)} \begin{bmatrix} \boldsymbol{p} \\ \boldsymbol{q} \\ \boldsymbol{r} \end{bmatrix} \tag{7.19}$$

Base Euler Angle Representation with Accelerometer Data:

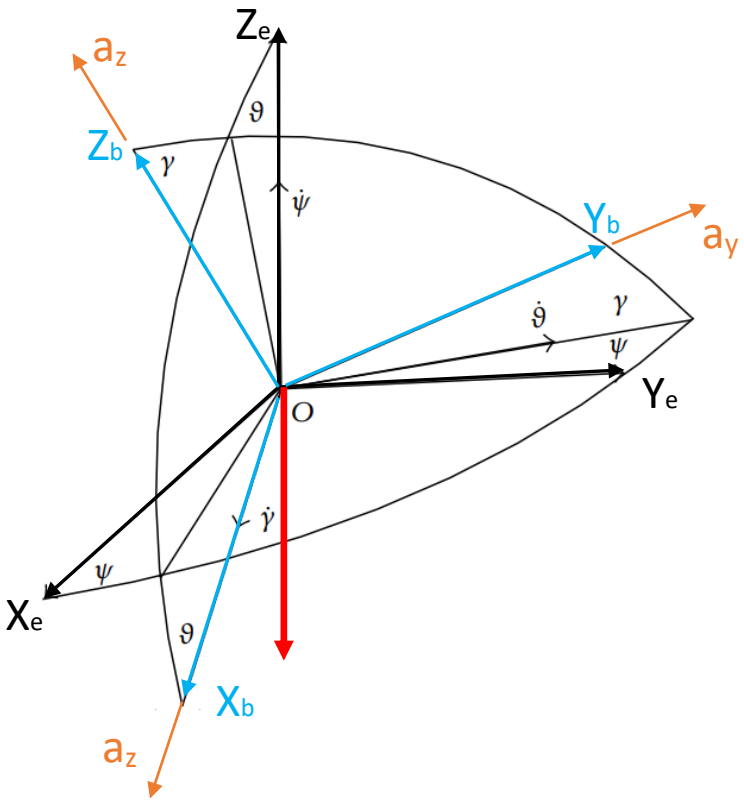


Figure 7.8: Gravity force representation on the Base Frame.

The accelerometer within IMU measures the acceleration of the attached object in 3D, additionally, due to the physical presence of gravity force, accelerometer is subjected to gravitational acceleration even at the stationary position. This property can be utilized for determination of pitch and roll degrees of the system. On Figure 7.8, the gravitational acceleration is represented as it coincided with the Earth Frame Z-Axis, however its measurement done on the Base Frame. Using obtained equation 7.10, the proper formulation between gravitational acceleration and the measured acceleration can be obtained. On equation 7.20, both equation 7.10 and its transpose demonstrated.

$$R_{e-b}(\psi,\vartheta,\gamma) = R_{e-1}(Z,\psi) * R_{1-2}(Y,\vartheta) * R_{2-b}(X,\gamma)$$
$$R_{b-e}(\psi,\vartheta,\gamma) = R_{b-2}(X,\gamma) * R_{2-1}(Y,\vartheta) * R_{1-e}(Z,\psi) \quad (7.20)$$

If the acceleration measurements on, X, Y, and Z directions of IMU represented as a_x, a_y, and a_z, respectively. Using equations on 7.20 following formulation between IMU measurements and gravitational acceleration can be obtained as given on equation 7.21.

$$a) \quad \begin{bmatrix} 0 \\ 0 \\ -g \end{bmatrix} = R_{e-b}(\psi,\vartheta,\gamma) * \begin{bmatrix} a_x \\ a_y \\ a_z \end{bmatrix}$$
$$b) \quad \begin{bmatrix} a_x \\ a_y \\ a_z \end{bmatrix} = R_{b-e}(\psi,\vartheta,\gamma) * \begin{bmatrix} 0 \\ 0 \\ -g \end{bmatrix} \quad (7.21)$$

Using equation 7.21 (b), equation 7.22 can be obtained explicitly.

$$\begin{bmatrix} a_x \\ a_y \\ a_z \end{bmatrix} = \begin{bmatrix} c(\psi)c(\theta) & s(\psi)c(\theta) & -s(\theta) \\ c(\psi)s(\theta)s(\gamma) - s(\psi)c(\gamma) & s(\psi)s(\theta)s(\gamma) + c(\psi)c(\gamma) & c(\theta)s(\gamma) \\ c(\psi)s(\theta)c(\gamma) + s(\psi)s(\gamma) & s(\psi)s(\theta)c(\gamma) - c(\psi)s(\gamma) & c(\theta)c(\gamma) \end{bmatrix} * \begin{bmatrix} 0 \\ 0 \\ -g \end{bmatrix} \quad (7.22)$$

With matrix multiplication following equation 7.23 is obtained.

$$\begin{bmatrix} \boldsymbol{a_x} \\ \boldsymbol{a_y} \\ \boldsymbol{a_z} \end{bmatrix} = \begin{bmatrix} \boldsymbol{s(\theta) * g} \\ \boldsymbol{-c(\theta)s(\gamma) * g} \\ \boldsymbol{-c(\theta)c(\gamma) * g} \end{bmatrix} \quad (7.23)$$

As shown on the equation 7.23, the accelerometer measurements can be used to estimate the pitch (θ) and roll (γ) angles at the stationary condition. However, yaw (ψ) rotation angle can not be estimated, since the gravitational acceleration is in the same direction as the yaw rotation axis. From equation 7.23, θ and γ can be formulated as shown on equation 7.24.

$$\gamma = atan2(a_y, a_z)$$
$$\theta = atan2(a_x, \sqrt{{a_y}^2 + {a_z}^2}) \quad (7.24)$$

Using Both Accelerometer and Gyro Measurements for Base Euler Angle Estimation:

With above equations 7.19 and 7.24, it can be seen that there are two different formulations for estimating the orientation of the IMU and the Base Frame.

The formulation with gyro data is exact, however it involves integration process, therefore noise and drift should be handled carefully, and if slight drift cannot be eliminated the error accumulates.

The formulation with accelerometer does not use integration process, however the formulation is restricted to the case where the IMU is subjected only to the gravitational acceleration. If there is acceleration apart from gravitational acceleration, the result only approximates the Base Frame orientation.

For better orientation estimations, both the gyro raw data and the accelerometer data are used with fusion algorithms, which may be Kalman Filters or Complementary Filters [15].

MPU 6050 has an internal digital motion processor called DMP, which does mentioned integration with fusion algorithm. Although, accelerometer and gyro are adequate for absolute estimation of θ *(pitch angle)* and γ *(roll angle)*, they are not adequate for absolute estimation of ψ *(yaw angle)*. For absolute yaw angle estimation of the Base Frame, an additional Magnetometer should be used. Due to the lack of Magnetometer, there is a slight drift with the yaw angle estimation in this project, however it should be noted that this drift is measured to be around 1º in 5 minutes.

In this project, internal DMP unit's ZYX Euler Angle estimations are used.

7.1.4.2 Estimating Position of the Base in Earth Frame (x, y, z)

With IMU, acceleration data that are obtained on the Base Frame, can be projected onto the Earth Frame with the help of rotation matrices. On equation 7.25, this formulation can be seen.

$$\frac{d^2}{dt^2}\begin{bmatrix} x \\ y \\ z \end{bmatrix} = R_{e-b}(\psi, \vartheta, \gamma) * \begin{bmatrix} u \\ v \\ w \end{bmatrix} \quad (7.25)$$

It should be noted that, in practice the similar noise and drift also affects this double integration process. In practice, there are methods for position tracking. One example of this method is the usage of fusion algorithms with computer vision processes and IMU data. Another example is usage of GPS and human input to correct the IMU data. Another example is using gait analyzes, where zero-velocity detection algorithms are used if there is a motion pattern on the IMU, such as foot movement when walking [16]. However, in literature it is noted that successful position tracking with only a low-cost IMU is not reported [17].

7.1.5 Inverse Kinematic for Gimbal Motors

In this project, the camera is attached to the Platform Frame. Therefore, desired orientation of the platform should be achieved with the 6 Rotation matrices given on the equation 7.26.

$$R_{e-p}(\psi,\vartheta,\gamma,\alpha_1,\alpha_2,\alpha_3) = R_{e-1}(Z,\psi) * R_{1-2}(Y,\vartheta) * R_{2-b}(X,\gamma) * R_{b-G1}(Z,\alpha_1) * R_{G1-G2}(Y,\alpha_2) * R_{G2-p}(X,\alpha_3) \tag{7.26}$$

The desired orientation of the platform can be defined in Earth Frame as given on the equation 7.27. Where β represents desired yaw angle of the platform, ζ represents desired pitch angle of the platform, and η represents desired roll angle of the platform. The orientation of the platform with 6 Rotations should be equal to equation 7.27, as given on the equation 7.28.

$$R_{Desired_{e-p}}(\beta,\zeta,\eta) = R(Z,\beta_{Desired}) * R(Y,\zeta_{Desired}) * R(X,\eta_{Desired}) \tag{7.27}$$

$$\boldsymbol{R_{Desired_{e-p}}(\beta,\zeta,\eta) = R_{e-p}(\psi,\vartheta,\gamma,\alpha_1,\alpha_2,\alpha_3)} \tag{7.28}$$

The purpose of the inverse kinematic is to find the appropriate α_1, α_2 and α_3 angles of the gimbal motors. Therefore, by eliminating, Base Frame orientations from the equation 7.28, the equation can be reduced to only motor angles as given on the equation 7.29. Here it should be noted, the transpose property of the rotation matrices have been used.

$$R_{Desired_{b-p}}(\alpha_1,\alpha_2,\alpha_3) = R_{b-2}(X,\gamma) * R_{2-1}(Y,\vartheta) * R_{1-e}(Z,\psi) * R_{Desired_{e-p}}(\beta_{Desired},\zeta_{Desired},\eta_{Desired}) \tag{7.29}$$

On equation 7.15, rotation sequence from base to platform was given, below it is shown to remind.

$$\boldsymbol{R_{b-p}(\alpha_1, \alpha_2, \alpha_3) = R_{b-G1}(Z, \alpha_1) * R_{G1-G2}(Y, \alpha_2) * R_{G2-p}(X, \alpha_3)}$$

On equation 7.30, explicit representation of the equation 7.15 can be seen.

$$R_{Desired\,b-p}(\alpha_1, \alpha_2, \alpha_3) =$$

$$\begin{bmatrix} c(\alpha_2)c(\alpha_1) & s(\alpha_3)s(\alpha_2)c(\alpha_1) - c(\alpha_3)s(\alpha_1) & c(\alpha_3)s(\alpha_2)c(\alpha_1) + s(\alpha_3)s(\alpha_1) \\ c(\alpha_2)s(\alpha_1) & s(\alpha_3)s(\alpha_2)s(\alpha_1) + c(\alpha_3)c(\alpha_1) & c(\alpha_3)s(\alpha_2)s(\alpha_1) - s(\alpha_3)c(\alpha_1) \\ -s(\alpha_2) & s(\alpha_3)c(\alpha_2) & c(\alpha_3)c(\alpha_2) \end{bmatrix} \quad (7.30)$$

For simplicity a rotation matrix can be defined as given on the equation 7.31.

$$R = \begin{bmatrix} R_{11} & R_{12} & R_{13} \\ R_{21} & R_{22} & R_{23} \\ R_{31} & R_{32} & R_{33} \end{bmatrix} \quad (7.31)$$

With the usage of equation 7.30 and 7.31, desired α_1, α_2 and α_3 angles can be found as given on the equation 7.32 [3].

$$\boldsymbol{\alpha_1 = atan2(R_{21}, R_{11})}$$

$$\boldsymbol{\alpha_2 = atan2\left(-R_{31}, \sqrt{{R_{32}}^2 + {R_{33}}^2}\right) or\ \alpha_2 = atan2\left(-R_{31}, \sqrt{{R_{11}}^2 + {R_{21}}^2}\right)} \quad \textbf{(7.32)}$$

$$\boldsymbol{\alpha_3 = atan2(R_{32}, R_{33})}$$

With obtained motor angles, appropriate orientation of the platform frame can be achieved, as shown on the Figure 7.9.

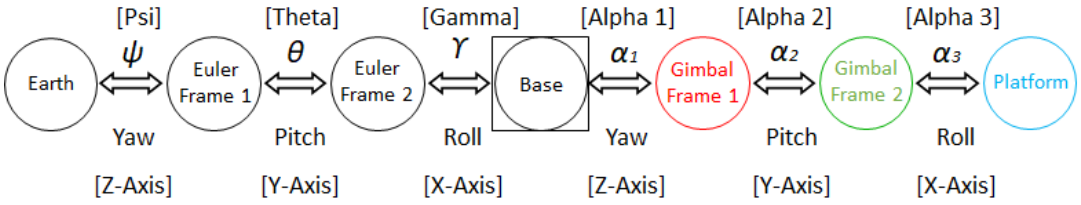


Figure 7.9: Reference frames from Earth Frame to Platform Frame.

7.2 Physical Design Model of the Gimbal System

While starting the project drawings, it was started with a first study as seen in Figure 7.10. The main purpose in this design was a design that showed what kind of design should be for adjusting the arms and adjusting the 3 axis.

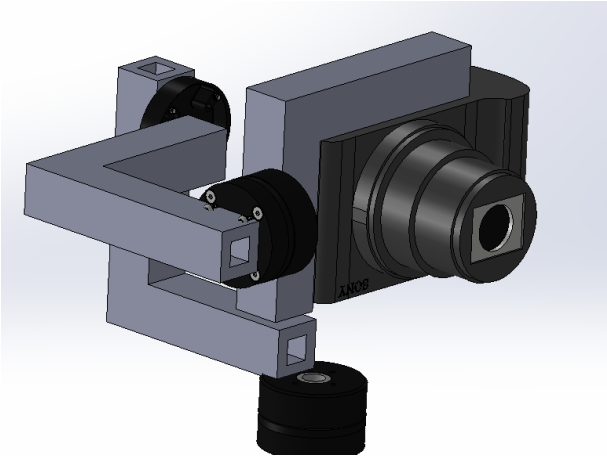


Figure 7.10 : Design 1.

The developed design considers factors such as usability, flexibility of the part, ease of manufacturing, and cost, resulting in a design similar to the one presented in Figure 7.11.

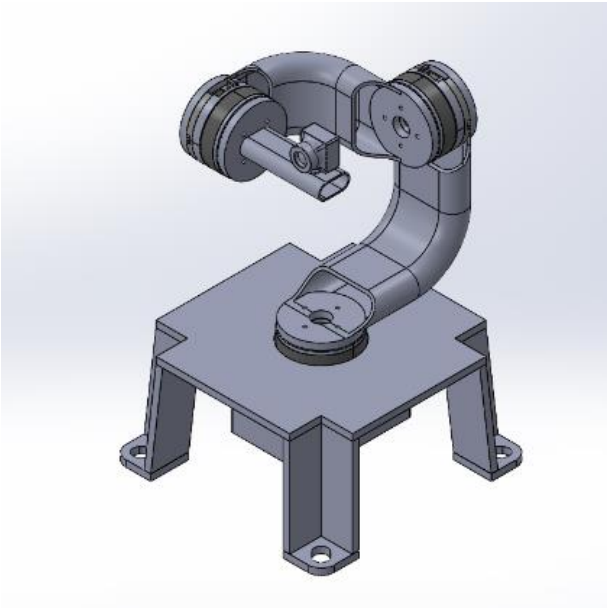


Figure 7.11: Design 2.

Later, it was observed that there was deviation in the axes as seen in Figure 7.11. In order to solve this issue, the camera was adjusted to be aligned with the center of the axes, as shown in Figure 7.12. Furthermore, in order to avoid unnecessary weight caused by thick supporting arms, changes were made by reducing and slimming down Arm 1 and Arm 2, as can be seen when comparing Figure 7.11 and Figure 7.12.

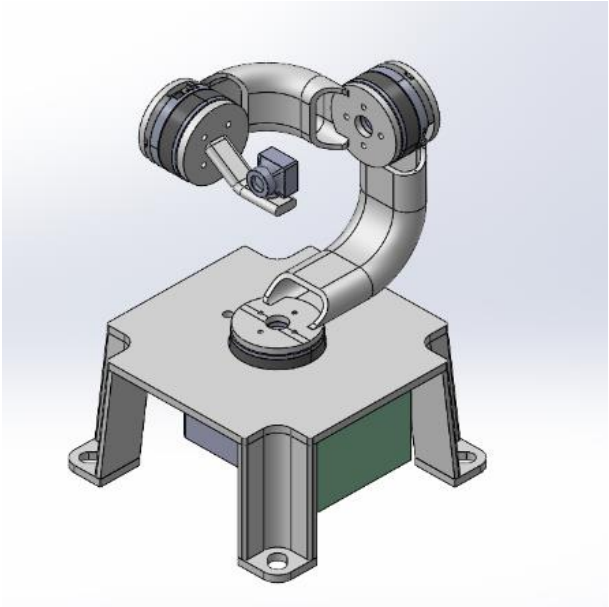


Figure 7.12: Design 3

A more useful camera with better resolution and image quality was selected, which led to changes in the design. As shown in Figures 7.13, 7.14, 7.15, and 7.16 designs were made to look at different axes, both with and without the camera's back adhesive part

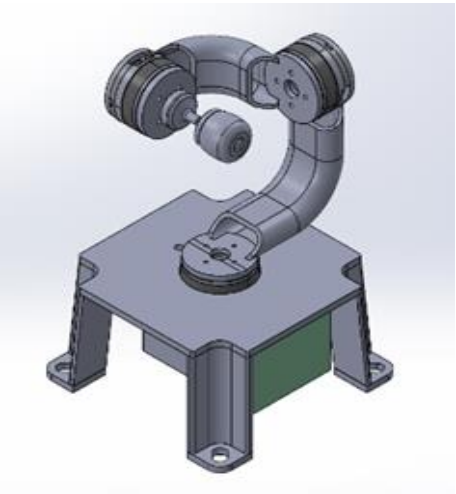


Figure 7.14: Design 4.

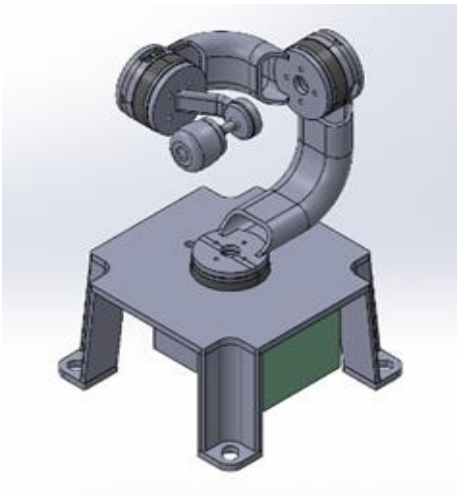


Figure 7.13: Design 5.

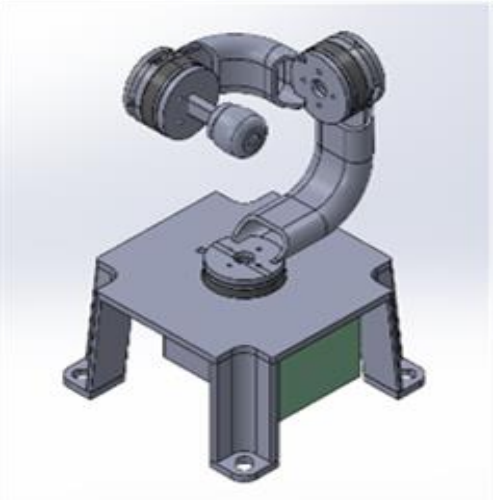


Figure 7.16: Design 6.

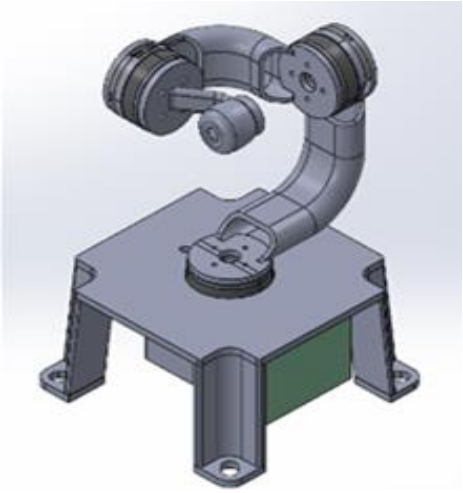


Figure 7.15: Design 7.

However, as seen in Figures 7.13 and 7.15, there was a deviation from the orthogonal position when using the attachment at the back of the camera. Therefore, the designs without the attachment, Figure 7.15 and 7.17, were continued. Later on, due to factors such as usability, the design was further developed and presented in Figure 7.17. In this design, the focus was on securing the camera and ensuring its stability. The box was preferred for the protection and concealment of the electronic components. A separate plinth was made suitable for the volume of the box.

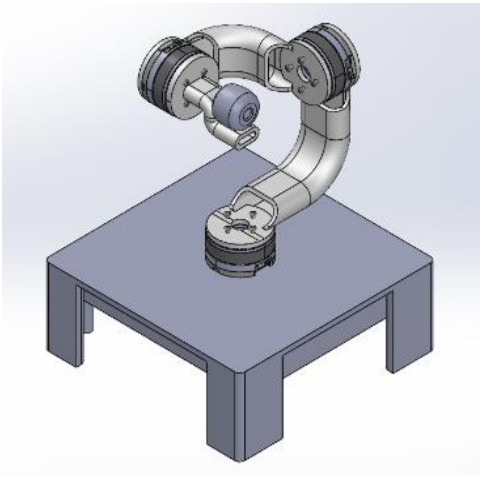


Figure 7.18: Design 8.

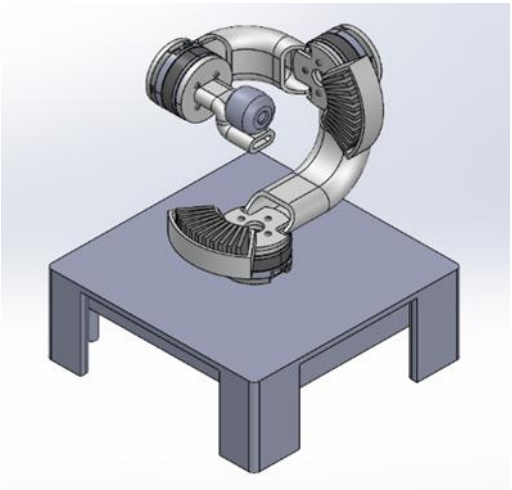


Figure 7.17: Design 9.

Continuing from the design presented in **Figure 7.17,** it was finalized as the ultimate design. In order to reduce the torque on the motor and balance the load, weight arms were added. As shown in **Figure 7.18,** the weight arms were mounted on the arms and each contained 10g weights. The center of mass and load balancing settings were adjusted accordingly.

The design presented in Figure 7.18 was selected as the final design. Images of the design from different angles are provided in Figure 7.19, 7.20, and 7.21.

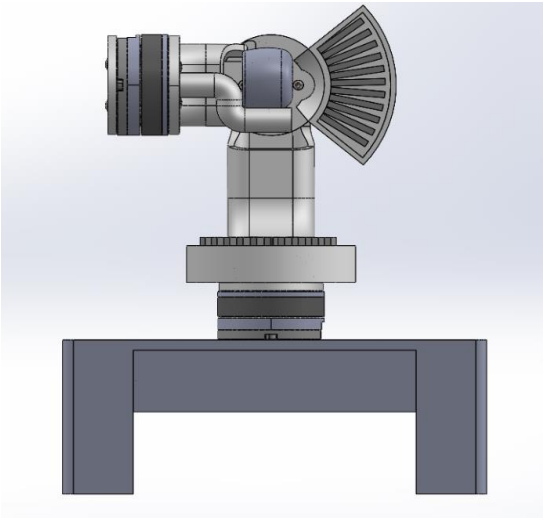


Figure 7.19: Design 9- Z Axis

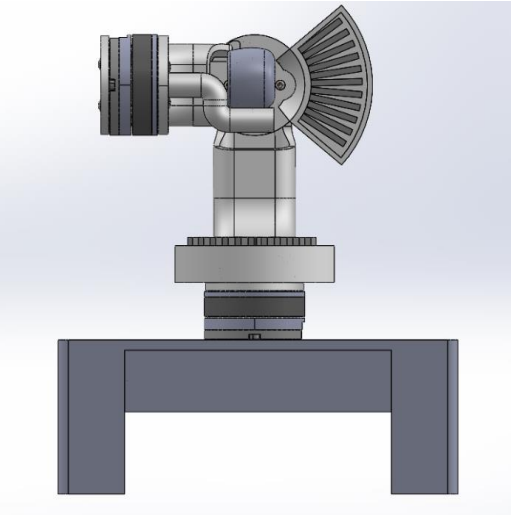


Figure 7.20: Design 9- Z Axis

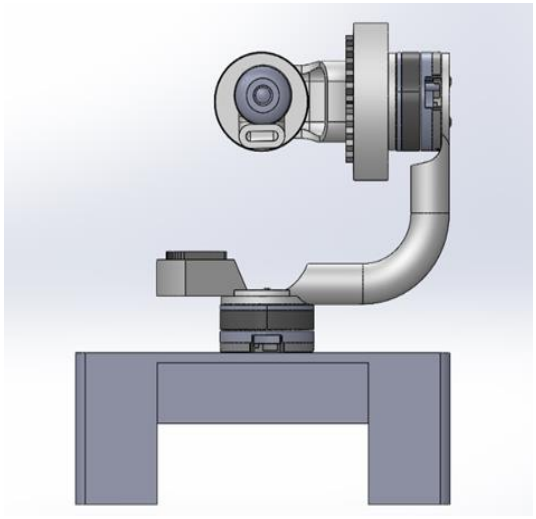


Figure 7.21: Design 9- X Axis

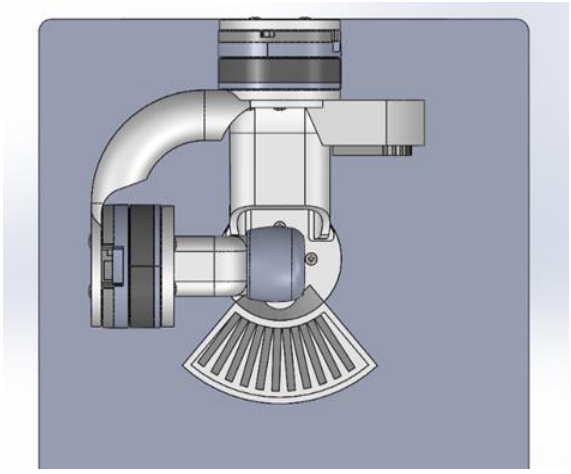


Figure 7.22: Design 9- Y Axis

7.3 Hardware and Digital Twin Design and Configuration

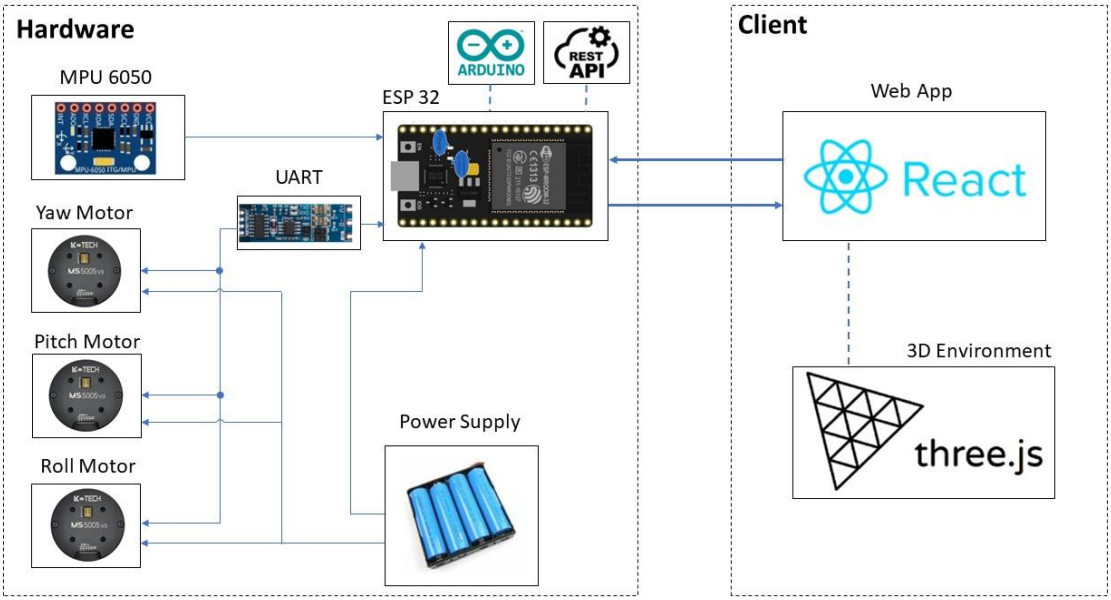


Figure 7.23: System architecture diagram.

The electronic materials used in gimbal manufacturing are as follows:

- NODEMCU ESP-32S
- MPU6050 Sensor
- UART-TTL to RS485 Converter
- MS5005V3 BLDC Motors
- Voltage Regulator
- Power Source

7.3.1 NODEMCU ESP-32S

In this study, the NodeMCU ESP-32S microcontroller was employed. The NodeMCU ESP-32S is a development board developed by NodeMcu specifically for evaluating the ESP-WROOM-32 module. It is based on the ESP32 microcontroller, which combines Wifi, Bluetooth, Ethernet, and Low Power support in a single chip. NodeMCU is an opensource platform for Internet of Things (IoT) applications that utilizes the Lua Scripting language.

Features of the NodeMCU ESP-32S include:

- Opensource Internet of Things (IoT) platform.
- Easy programmability, allowing for rapid development and prototyping.

- Cost-effective and straightforward to implement in projects.
- Integrated Wi-Fi capability, enabling wireless communication for IoT applications.

The decision to utilize the NodeMCU ESP-32S microcontroller in this project was motivated by several factors. Firstly, its compatibility with the Arduino Integrated Development Environment (IDE) makes coding straightforward and convenient. Secondly, its built-in WiFi capability enables seamless data transfer over a wireless network, eliminating the need for external antennas. The ESP32 WiFi module supports multiple modes of operation, including station, access point, or both. In this project, the ESP32 is configured as a station, as depicted in the diagram below. It establishes a connection with a router and facilitates the transfer of gimbal data to connected devices over the network.

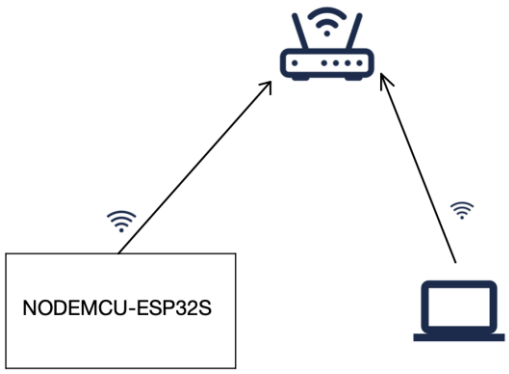


Figure 7.24: ESP32 as Station connected to Router, transferring gimbal data to connected devices.

The pin functionalities of the Node MCU ESP-32S microcontroller used in the system are illustrated in Figure X. GPIO22, GPIO21, GND, and 3.3V pins shown in the diagram are used for connecting the MPU6050 sensor, while GPIO1, GPIO3, and GND pins are used for sending commands to motors and receiving data from them.

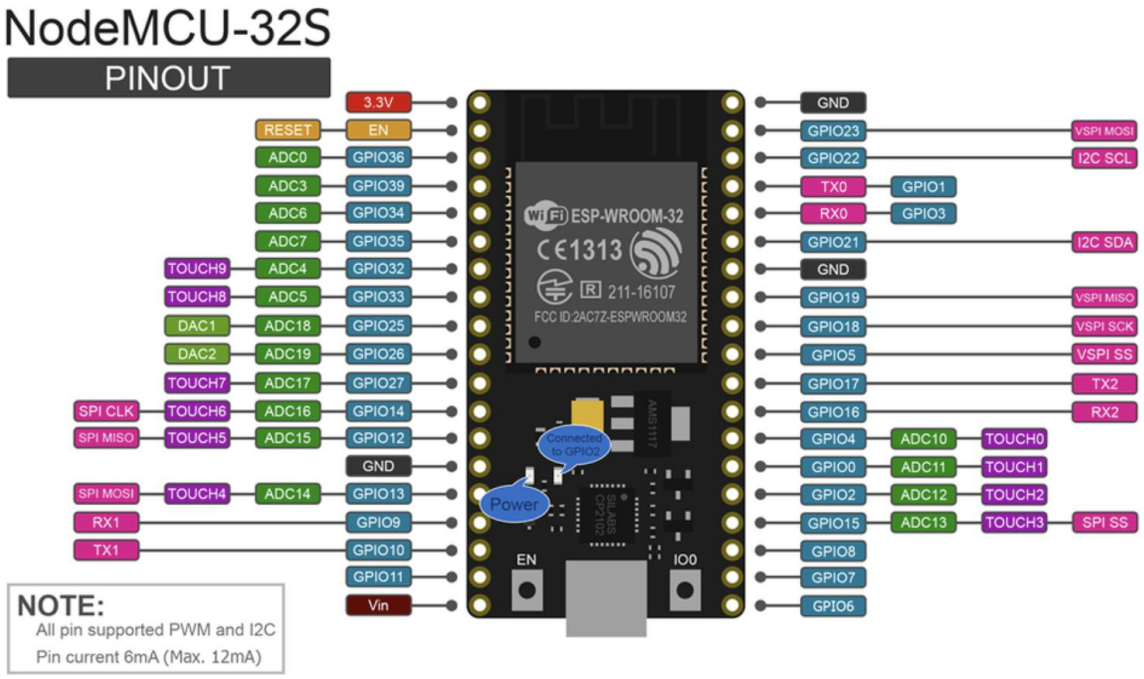


Figure 7.25: NodeMCU ESP-32S pinout.

Power supply to the microcontroller is provided through the "Vin" pin with a voltage regulator, supplying five volts. The "GND" pins shown in the diagram are ground pins.

7.3.2 MPU6050 Sensor

The MPU6050 is a module that consists of a 3-axis accelerometer and a 3-axis gyroscope. The gyroscope measures rotational velocity in units of rad/s, which represents the change in angular position over time along the X, Y, and Z axes (roll, pitch, and yaw). This enables the determination of the orientation of an object. On the other hand, the accelerometer measures acceleration, which is the rate of change of an object's velocity. It can sense static forces like gravity as well as dynamic forces like vibrations or movements. The MPU6050 measures acceleration along the X, Y, and Z axes. In the diagram below, the image of the MPU6050 sensor and the pins on it are shown.

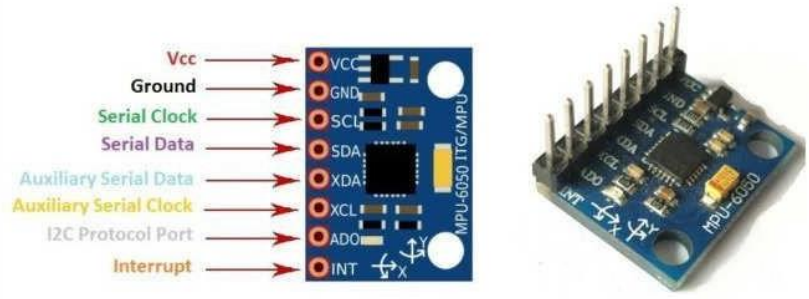


Figure 7.26: MPU6050 pinout.

The pins shown in the diagram, "VCC", "GND", "SCL", and "SDA", are used in the project. The "VCC" pin serves as the power supply pin for the sensor and can be powered with either 3V or 5V, which is connected to the "3.3V" pin on the microcontroller. The "GND" pin is the ground pin, which is connected to any "GND" pin on the microcontroller. The "SCL" pin, also known as Serial Clock, and the "SDA" pin, also known as Serial Data, are connected to "GPIO22" and "GPIO21" pins on the microcontroller, respectively. These pins facilitate I2C communication between the microcontroller and the sensor, allowing data to be retrieved from the sensor. Other pins on the MPU6050 are not used in this project. The connection between the microcontroller and the sensor is shown in the diagram below.

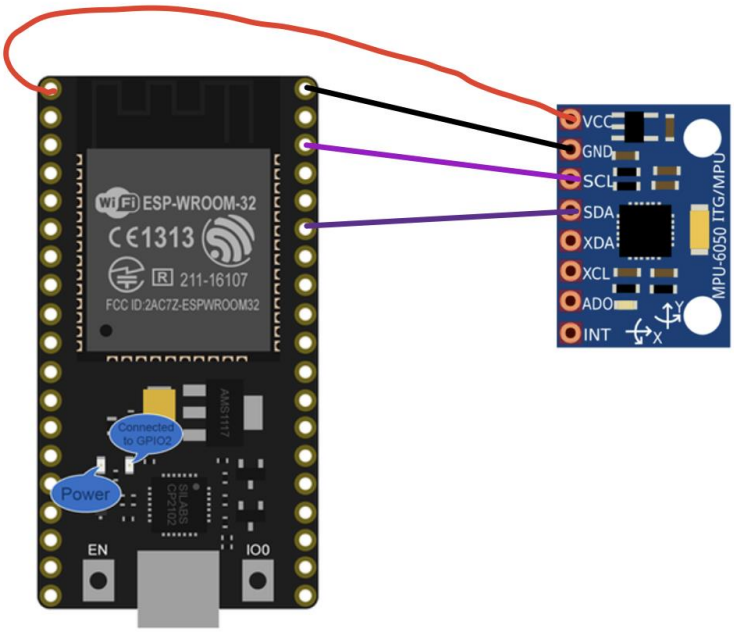


Figure 7.27: The connection between the microcontroller and the sensor.

7.3.3 MS5005V3 BLDC Motors

Upon examining the gimbals currently available on the market, it has been observed that the motors used in gimbals are brushless DC motors, also known as BLDC motors. When studying gimbal works carried out with servo motors, it has been observed that the resulting product is not in line with the nature of a gimbal, which is intended to obtain smooth and silky images from the camera attached to it. Therefore, it has been decided to use brushless DC motors to achieve this desired result.

A brushless DC electric motor (BLDC), also known as an electronically commutated motor, is a synchronous motor using a direct current (DC) electric power supply. It uses an electronic controller to switch DC currents to the motor windings producing magnetic fields which effectively rotate in space and which the permanent magnet rotor follows. The controller adjusts the phase and amplitude of the DC current pulses to control the speed and torque of the motor.

Brushless DC motors use an electronic servo system instead of mechanical commutator contacts. An electronic sensor detects the rotor angle and controls semiconductor switches, such as transistors, to switch current through the windings, either reversing the direction of the current or turning it off at the correct angle to create torque in one direction using electromagnets. The absence of sliding contacts in brushless motors reduces friction and prolongs their lifespan, limited only by the lifetime of their bearings.

Unlike brushed DC motors that develop maximum torque when stationary and linearly decrease as velocity increases, brushless motors can overcome some limitations of brushed motors, including higher efficiency and lower susceptibility to mechanical wear. However, brushless motors may require more complex and expensive control electronics and may be less rugged.

A typical brushless motor has fixed armature with rotating permanent magnets, eliminating issues associated with connecting current to a moving armature. An electronic controller replaces the commutator assembly of brushed DC motors, continuously switching the phase to the windings to keep the motor running. The controller uses a solid-state circuit instead of a commutator system for timed power distribution.

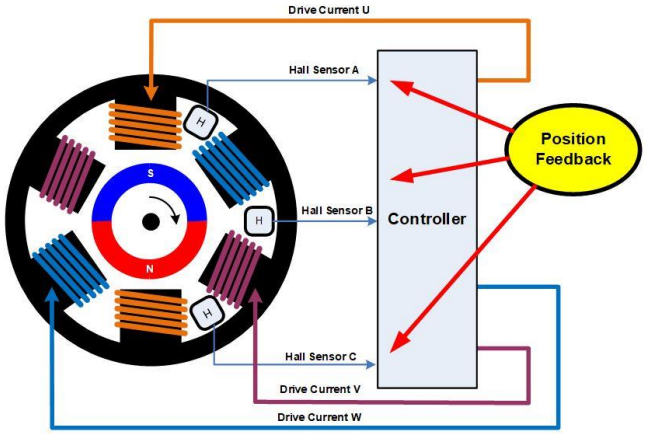


Figure 7.28: Controller and Brushless Motor.

The windings of a brushless motor can be connected in either a 'star' or a 'delta' configuration, with three wires connecting to the motor, and the drive technique and waveform being identical in both cases. Motors with three phases can be constructed with different magnetic configurations, referred to as poles. The simplest 3-phase motors have two poles, with the rotor having only one pair of magnetic poles - one North and one South. Motors can also be built with more poles, requiring additional magnetic sections in the rotor and more windings in the stator. Higher pole counts can result in higher performance, although very high speeds are typically better achieved with lower pole counts.

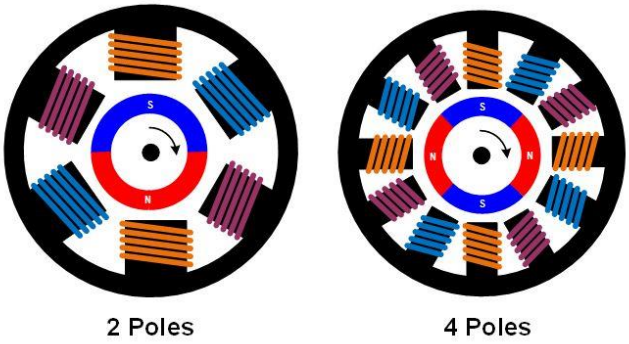


Figure 7.29: BLDC motors with different pole numbers.

As mentioned, there are specific methods for driving and controlling BLDC motors. Considering both time and R&D costs, it was decided that the motor and its driver/controller should be integrated together. As a result of research, the motor selected for this project is the MS5005V3 motor from a company called LK-Tech in China. The technical specifications and features of the motor are provided under the Appendix section. This motor is sold integrated with its controller, magnetic encoder,

and driver. The motors used in the project implement the RS485 communication protocol to execute the commands given. Three of these motors are used in the project.

Figure 7.30: MS5005V3 motor.

A BUS structure has been established using the RS485 protocol, allowing the motors to be connected to each other without the need for individual microcontrollers, and only one motor (yaw motor) is connected to the microcontroller. Each motor has been assigned an ID through the ID switches on them, enabling different commands to be sent to each motor using these IDs.

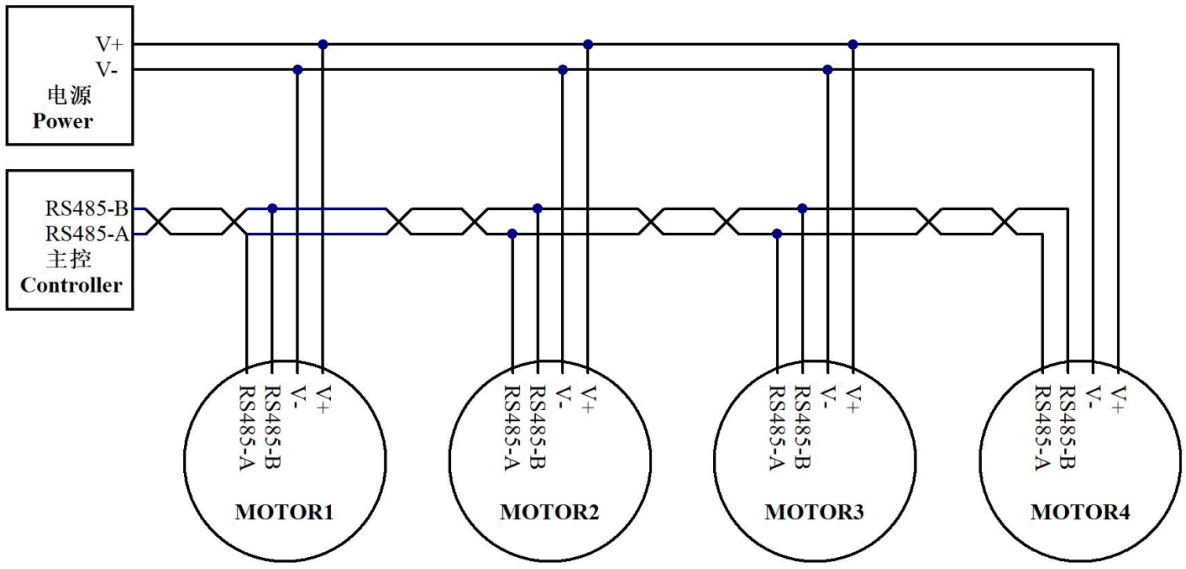


Figure 7.31: RS485-BUS Structure.

7.3.4 The other electrical components

As mentioned, commands are sent to the motors in the project using the RS485 protocol. However, the microcontroller used does not support any RS485 pins. Therefore, a UART-TTL to RS485 converter is used to send commands to the motors. The desired command to be sent from the microcontroller is transmitted via UART and then forwarded to the motors in RS485 format through the converter. The "GPIO1", "GPIO3", and "GND" pins on the microcontroller are used for this purpose. The "GPIO1" pin is the UART TX pin, used for data transfer, and is connected to the pin labeled as TXD on the converter. The "GPIO3" pin is the UART RX pin, used for data reception, and is connected to the pin labeled as RXD on the converter.

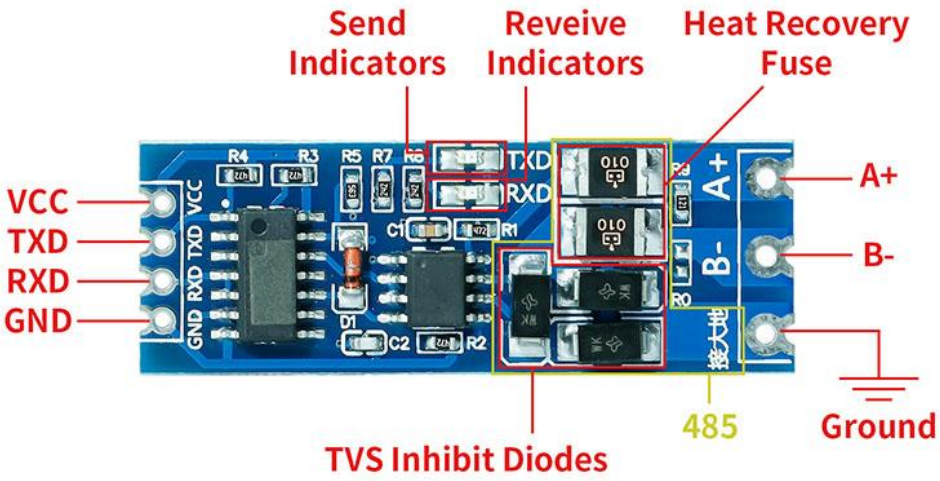


Figure 7.32: UART-TTL to RS485 Converter.

As seen in the diagram, the converter also has "GND" and "VCC" pins. The "VCC" pin is powered by a 5-volt voltage regulator, while the "GND" pin is connected to the ground pin of the voltage regulator. Below is a visual representation of the connections between the microcontroller, converter, and motors.

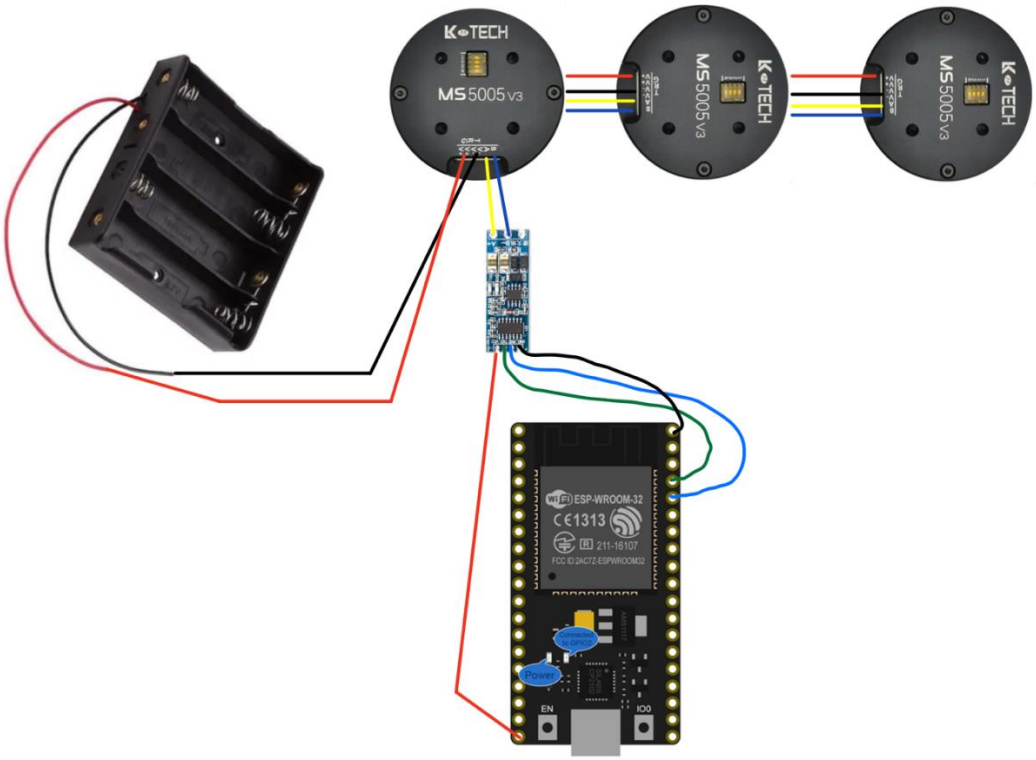


Figure 7.33: Connection between ESP32, converter and motors.

A battery is used for power supply in the entire electrical system, providing a total of 22 volts. This voltage value is suitable for the motors, but it is too high for the microcontroller and converter. Therefore, a voltage regulator is used, which takes in 22V and outputs 5V.

Figure 7.34: Voltage regulator.

A combined image of all the described electrical components is provided below.

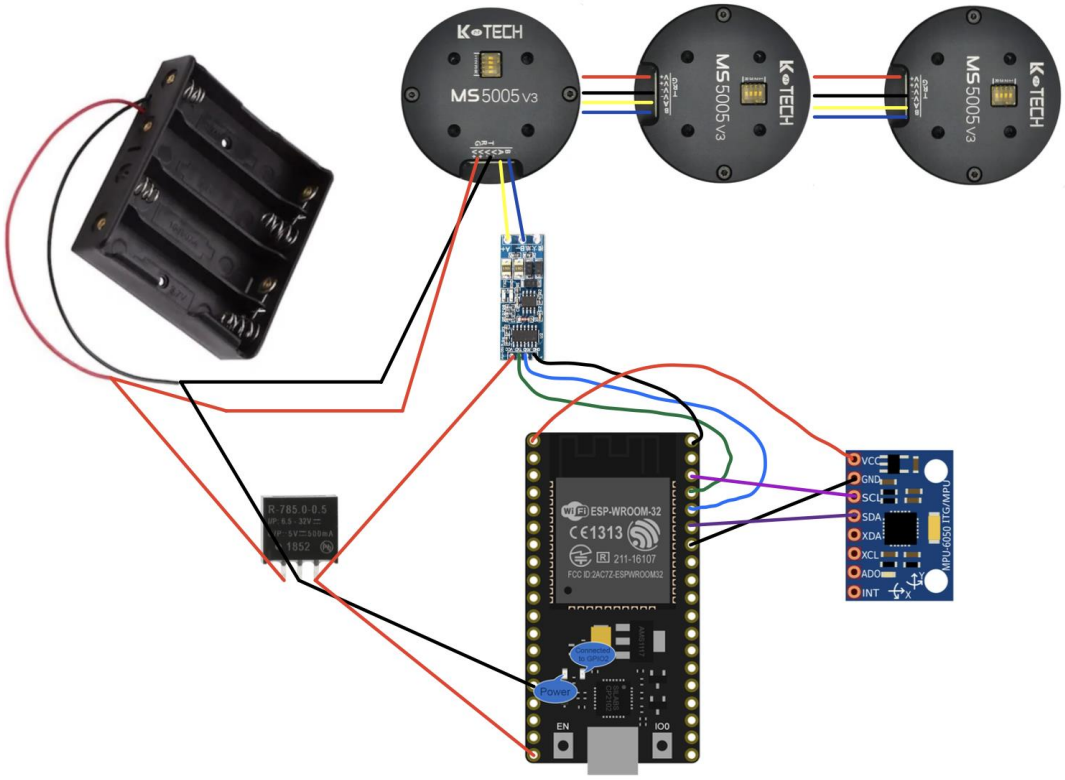


Figure 7.35: The entire electrical setup is shown.

7.3.5 Microcontroller Programming

The microcontroller was programmed using Arduino IDE. The codes written in Arduino IDE are provided under the Appendix section. The libraries used are as follows:

- HardwareSerial library

  This library is specifically designed for ESP32 and enables command sending over UART.

- Math library

  This library enables mathematical operations to be performed within the code.

- I2Cdev library

  This library is required for communication between the sensor and the microcontroller.

- MPU6050_6Axis_MotionApps20 library

  This library is required to use the sensor with the microcontroller.

- KickMath library

  A library for performing a few simple mathematical calculations for use with arrays.
- BasicLinearAlgebra library

  A library for representing matrices and doing matrix math on Arduino.
- WiFi library

  Enables network connection (local and internet) using the esp32.
- ESPAsyncWebServer library

  Async HTTP and WebSocket Server for ESP boards. Thanks to this library, the microcontroller is used as a server.
- ArduinoJson library

  A simple and efficient JSON library for embeded C. Thanks to this library, data can be transmitted in JSON format.

The microcontroller programming is done using the mentioned libraries. First, the microcontroller connects to a Wi-Fi network and publishes data under the digital twin heading, as explained in more detail. Then it receives data from the sensor to obtain the desired position for each motor. Commands are sent to the motors to go to a certain position. An example command sent to the motor is as follows:

3E A5 01 04 E8 00 28 23 00 4B

The given example command is in hexadecimal format, i.e., written in base-16. The first index "3E" is a fixed value. The second index "A5" indicates that position control is being performed on the motor, and the microcontroller understands that position data has been received when this value is seen. The third index "01" is the ID value that specifies which motor this command is sent to. The fourth index "04" is a fixed value. The fifth index "E8" is the accumulated hexadecimal value of the commands sent so far. The sixth index "00" indicates the direction in which the motor will rotate to reach the specified angle. If this value is "00", it means clockwise, and if it is "01", it means counterclockwise. The seventh and eighth indices are the indices indicating the position to which the motor will go. Here, the motor takes angle values from 0 to 359.99 degrees, and the received value is divided by 0.01 and written in hexadecimal

format to these indices. However, the last two digits of the hexadecimal value written correspond to the seventh index of the command, and the first two digits correspond to the eighth index. The ninth index of the command is a fixed value of "00". The tenth index "4B" is the value obtained by adding up the hexadecimal numbers received from the sixth index onwards.

7.4 Digital Twin

Digital twin is a virtual representation of a gimbal. There is a real-time wireless data flow between the gimbal and the digital twin, which allows the digital twin to mimic the movements of the gimbal. The digital twin is developed as a website using React.js. The website contains a 3D model of the gimbal, and the gimbal can be controlled as desired from the website, and the resulting movements are reflected both on the actual gimbal and on the 3D model within the website in real-time. The layers of the digital twin are shown below, consisting of two layers: hardware and client.

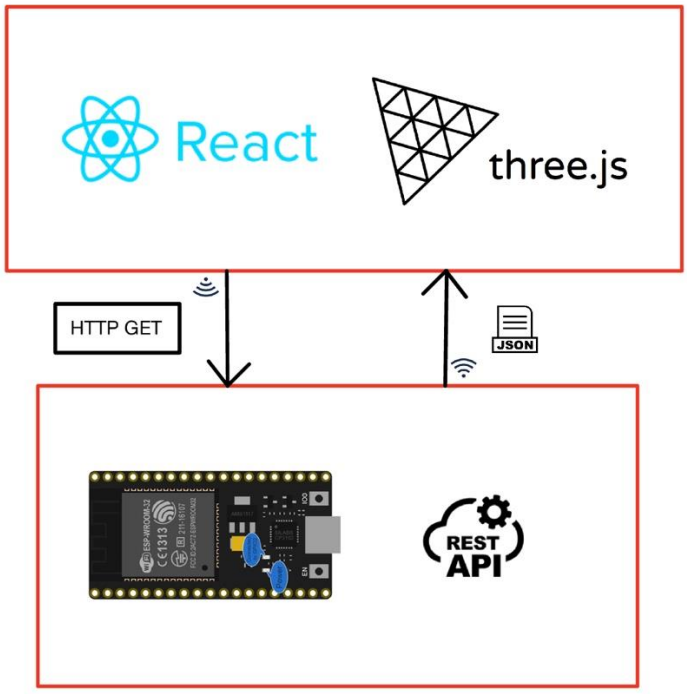


Figure 7.36: Layers of digital twin.

7.4.1 Hardware Side

ESP32 was used as a REST API. A REST API (also known as RESTful API) is an application programming interface (API or web API) that adheres to the constraints of the REST architectural style and enables interaction with RESTful web services. REST stands for representational state transfer.

An API is a set of definitions and protocols for building and integrating application software. It is often referred to as a contract between an information provider and an information user, defining the content required from the consumer (the call) and the content required by the producer (the response). For example, the API design for a weather service could specify that the user needs to supply a zip code and the producer will respond with a 2-part answer, including the high temperature and the low temperature. In other words, an API helps communicate the desired action or information to a computer or system so that it can understand and fulfill the request.

REST is a set of architectural constraints, not a protocol or a standard. API developers can implement REST in various ways. When a client makes a request via a RESTful API, it transfers a representation of the resource's state to the requester or endpoint. This representation is delivered in one of several formats using HTTP, such as JSON (JavaScript Object Notation), HTML, XML, Python, PHP, etc. Headers and parameters are also important in the HTTP methods of a RESTful API request, as they contain essential identifier information, including metadata, authorization, uniform resource identifier (URI), caching, cookies, and more. There are request headers and response headers, each with their own HTTP connection information and status codes.

In the project, the developed digital twin application sends requests to the ESP using the HTTP GET method. The ESP responds to this request by sending data in JSON format or status codes to the client to indicate whether the request has been fulfilled or not. These operations are carried out through the "ESPAsyncWebServer" library. The data received from the gimbal in JSON format is converted to a readable format using the "ArduinoJSON" library.

In JSON format, data consists of two components: key and value. The client can access the value information using the key value of the received data. Below is an example of data in JSON format.

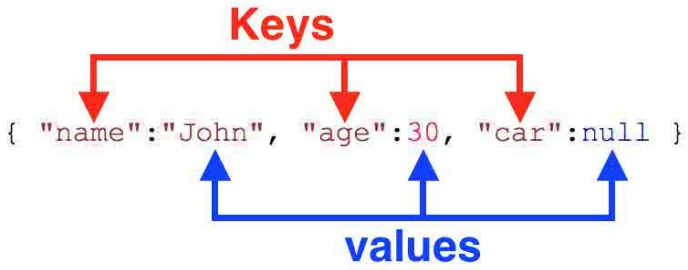


Figure 7.37: Example JSON data.

The WiFi module on the ESP connects to a nearby WiFi network, allowing the ESP to start a server. A client connected to the same network can access the ESP by using the ESP's IP address within the network and sending an HTTP GET request to request data or instruct the server to perform a certain action. The ESP responds to the client by sending the requested data or a status code indicating that the requested action has been completed. This way, communication between the ESP and the client is established.

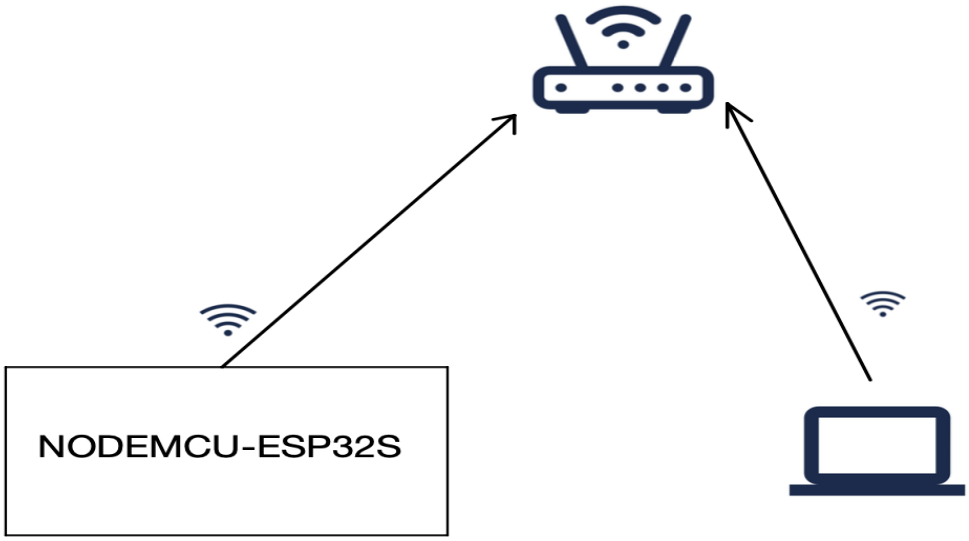


Figure 7.38: ESP-Client Communication.

7.4.2 Client Side

The client side of the project has been developed using React.js, an open-source JavaScript framework and library created by Facebook, and Three.js has been used for 3D environment.

React.js is utilized for building interactive user interfaces and web applications quickly and efficiently. It follows a component-based approach where reusable components are created, which are like independent Lego blocks that can be assembled to form the entire user interface of the application.

On the other hand, Three.js is a cross-browser JavaScript library and API used for creating and displaying animated 3D computer graphics in a web browser using WebGL. It allows for GPU-accelerated 3D animations using JavaScript, without the need for proprietary browser plugins, thanks to WebGL, a low-level graphics API specifically designed for the web.

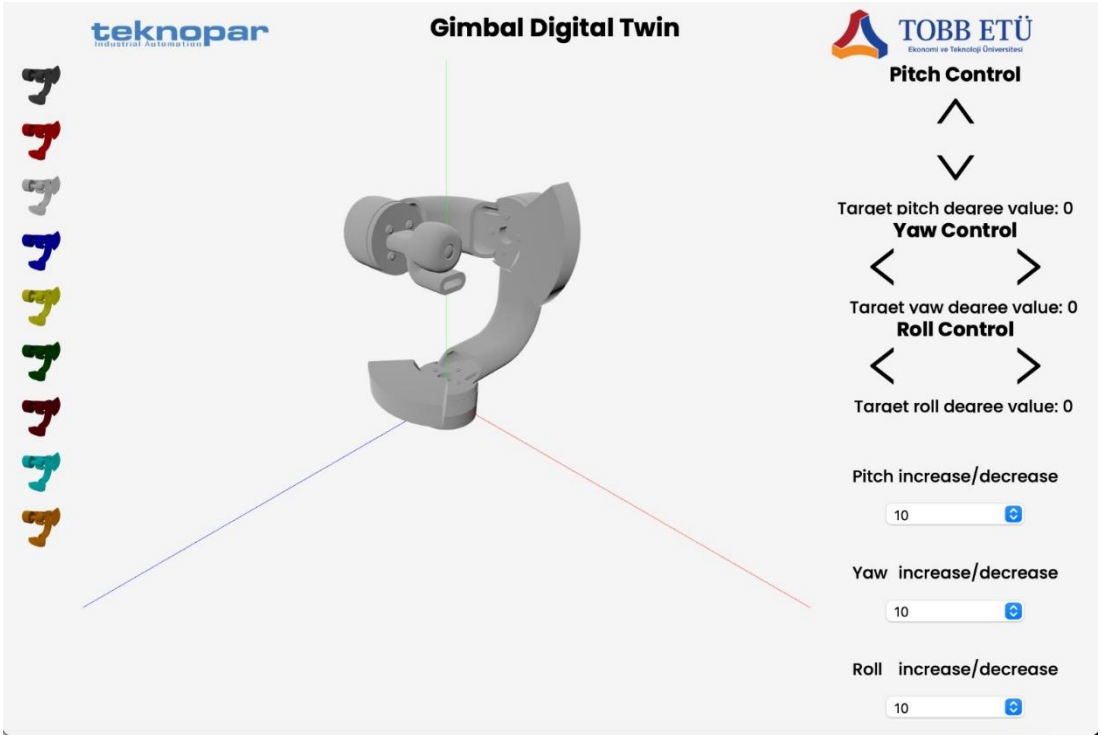


Figure 7.39: The screenshot of the Digital Twin Application.

On the client side, requests are sent to the API every 100 milliseconds to fetch the current state of the gimbal. The request is formatted as "http://ESP_IP/getAll", where ESP_IP is the IP address of the ESP module on the network. In response to this request, the ESP module sends the MPU6050 sensor data and motor position data in JSON format to the client. The client then updates the 3D model with the received data. Similarly, the client can control the gimbal's viewing angle, and the ESP module responds to these requests with a status code indicating whether the request was successful or not. A successful request is indicated by a status code of "200", but if the request fails, the status code may vary depending on the reason for the failure.

The application developed with React currently runs locally on the computer where it was developed. If the computer's connected network allows, the application can be

accessed with an IP address within the network. This way, other devices connected to the same network can also access the application.

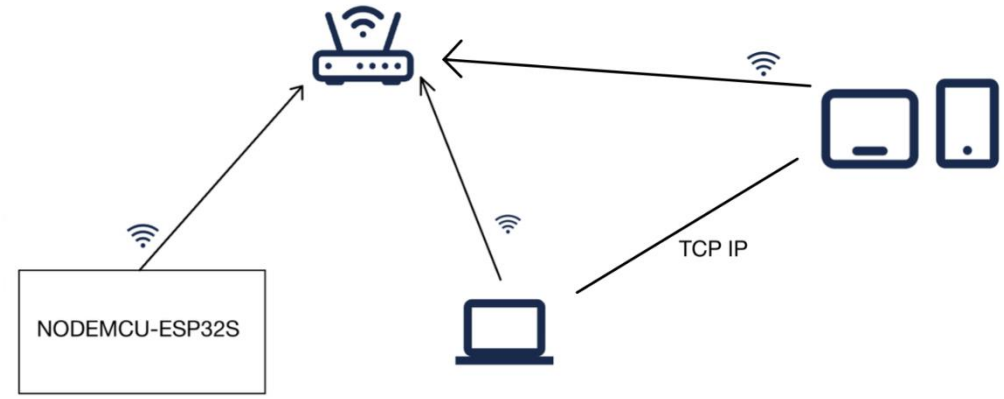


Figure 7.40: Diagram showing other devices on the network accessing the application.

8. VERIFICATION OF RESULTS

In this senior design project, a three-axis gimbal was constructed, and a stabilization experiment was witnessed by a camera to verify the performance of the proposed gimbal system, where the motors were used to execute to reverse movements in yaw, pitch, and roller axes.

Project goal was creating gimbal system that can communicate with its digital analogous and stabilize the scene while some kind of defending car road. With testing that is movement of the gimbal through a range of motions and observing its performance, it could be seen that the stabilization and communication were provided. After testing was completed data (simulated and experimental results) verified this. The scene remained the same. The video application and presentation proved that also. While simulation was working, the software tool was also a verifying document.

Testing and analysis results showed that any improvements or modifications can be made to the gimbal system for example using image processing while targeting, position obtaining targeting. It was an open project for future iterations.

9. BILL OF MATERIALS AND COST ESTIMATION

Table 9.1: Bill table.

Products	Quantity	Price per Unit	Price	Detail
Microcontroller	1	≈ ₺220	₺220	NodeMCU-32S ESP32 (WiFi+Bluetooth)
IMU Sensor	2	≈ ₺95	₺190	MPU6050 6 Axis Acceleration and Gyro Sensor
3D Printer Material	4	≈ ₺120	₺480	ABG 1.75mm Siyah PETG Filament
Motor BLDC	4	≈ ₺1600	₺6400	MS5005V3(RMD-S-5005) High-Accuracy servo BLDC motor for 3 axis handheld gimbal
Converter	3	≈ ₺45	₺135	UART TTL - RS485 Two-way Converter
IP camera	1	≈ ₺400	₺400	Wevolt X10 1080P Ultra Mini Wifi Kamera
Electrical Panel	1	≈ ₺130	≈ ₺130	G-765 156 x 180 x 52 mm (ABS)
Additional Expenses		≈ ₺1000	₺1000	NA
			₺8955	

From the table costs can be seen. These materials were evaluated and bought in accordance with each other to execute. Esp-32 microcontroller was bought due to its WiFi feature.4 BLDC motor was bought included one reserve motor in case of problem in of them. At 3D material and printing also helps were received from our academicians. UART converter was bought to provide communication between motors and Esp-32.

Additionally, an IP camera was bought to show the stabilization of scene supplied by motors. That costed 400 lira. Additionally, IP camera was bought considering these requirements:

- WiFi communication protocol
- built-in battery
- sufficient view

10.DISCUSSION AND CONCLUSION

In this senior design project, designing and developing a reliable and efficient mechanical stabilization device and its digital twin that can communicate, transfer data, and process simultaneously were aimed to be created for defense industry. The design process involved the selection of suitable components, including the motors, sensors and ensure optimal performance. The design team also had to consider factors such as weight, balance, and durability to ensure the device could handle the weight of cameras and other equipment.

During the development phase, the team encountered several challenges including mass unbalances, motor limitations, modelling, wireless data transfer and control mechanism problems. However, these challenges were overcome through continuous testing and working over the system.

The Gimbal successfully stabilized the camera. It works in synchrony with the developed digital twin of the gimbal. The user can direct the gimbal through the digital twin, and these directions can be seen on the 3D model on the developed application and on the actual gimbal.

Overall, the Gimbal project demonstrated the importance of planning, design, and testing in the development of mechanical and electronic devices. The project also highlighted the need for collaboration and teamwork to overcome challenges and deliver a successful product.

In upcoming works, the gimbal can be modified for other vehicles and industries. Using image processing and image recognition target clogging can be provided. With linear positioning, clogging and target following could be done in future projects.

REFERENCES

[1] – **Hilkert, J. M**, (2008), Inertially Stabilized Platform Technology Concepts and Principles. *IEEE Control Systems Magazine, 28(1): 26-46,* February 2008.

[2] – **Jie, M. and Qinbei, X.**, (2015), Four-axis Gimbal System Application Based on Gimbal Self-Adaptation Adjustment. *Proceedings of the 34th Chinese Control Conference*. China: Hangzhou, July 28-30.

[3] – **Lynch, K. M. and Park, F. C.**, (2017), *Modern Robotics: Mechanics, Planning and Control*. Cambridge: Cambridge University Press.

[4] – **Reis, M. F.**, (2018), *Line-of-Sight Stabilization and Tracking Control For Inertial Platforms.* (Master of Science Thesis). Coppe UFRJ (Instituto Alberto Luiz Coimbra de Pós-Graduação e Pesquisa em Engenharia) Department of Electrical Engineering, Rio deJanerio.

[5] – **Poyrazoğlu, E.,** (2017), *Detailed Modeling and Control of a 2-DOF Gimbal System.* (Master of Science Thesis). Middle East Technical University, The Graduate School of Natural and Applied Sciences, Ankara.

[6] – **Marsh, E.A.,** (2008). *Inertially Stabilized Platforms for SATCOM On-The-Move Applications: A Hybrid Open/Closed-Loop Antenna Pointing Strategy*. [Master of Science in Aeronautics and Astronautics Thesis, Massachusetts Institute of Technology]

[7] – **Ball, M.** (2021, Dec 03). *New UAV ISTAR Gimbal Launched*. Unmanned System Technology. Retrieved from: "https://www.unmannedsystemstechnology.com /2021/12/ new-uav-istar-gimballaunched/"

[8] – [GroundRig (a remote controlled Gimbal Car Camera Rig)]. Cinescopophilia. Retrieved from: "https://cinescopophilia.com/groundrig-a-remote-controlled-gimbal-car-camera-rig/"

[9] – **Saxena, A. and Sahay, B.**, (2005), *Computer Aided Engineering Design*. New Delhi: Anamaya Publishers.

[10] – **Waveshare,** (not available), *Flexible And Expandable Off-Road Tracked UGV, Multiple Hosts Support, With External Rails and ESP32 Slave Computer*. Retrieved from: "https://www.waveshare.com/ugv01.htm"

[11] – **Öztürk, T.,** (2010), *Angular Acceleration Assisted Stabilization of a 2-DOF Gimbal Platform.* (Master of Science Thesis). Middle East Technical University, The Graduate School of Natural and Applied Sciences, Ankara.

[12] – **Yao, W. and Hsueh, P. and Yeh, H.,** (2020), Dynamic Analysis and Stabilizing Control of Three-Axis Spherical Gimbal. *Journal of Systems and Control Engineering, Vol. 234(5) 609-621.* DOI: 10.1177/0959651819874559

[13] – **Sushchenko, O. and Goncharenko, A.** (2016), Design of Robust Systems for Stabilization of Unmanned Aerial Vehicle Equipment. *International Journal of Aerospace Engineering, Vol. 2016, Article ID 6054081, 10 pages.* DOI: https://doi.org/10.1155/2016/6054081

[14] – **Büyüksarıkulak, M. S.,** (2014), *Autopilot Design for a Quadrotor.* (Master of Science Thesis). Middle East Technical University, The Graduate School of Natural and Applied Sciences, Ankara.

[15] – **The MathWorks Inc.** (2023). MATLAB version: 9.13.0 (R2022b), Natick, Massachusetts: The MathWorks Inc., Retrieved from: https://www.mathworks.com/help/nav/ref/imufilter-system-object.html

[16] – **Skog, I., Händel, P. and Rantakokko, J.** (2010), Zero-Velocity Detection – An Algorithm Evaluation. *IEEE Transactions on Biomedical Engineering, Vol. 57, NO 11.* DOI: 10.1109/TBME.2010.2060723

[17] – **Llorach, G., Evans, A., Agenjo, J. and Blat, J.** (2014), Position Estimation with a Low-Cost Inertial Measurement Unit. *Information Systems and Technologies (CISTI), Portugal.* DOI:10.1109/CISTI.2014.6877049

[18] – **Li, L., & Chou, W.** (2011, July). Design and describe REST API without violating REST: A Petri net based approach. In *2011 IEEE International Conference on Web Services* (pp. 508-515). IEEE.

[19] – **Kousi, N., Gkournelos, C., Aivaliotis, S., Giannoulis, C., Michalos, G., & Makris, S.** (2019). Digital twin for adaptation of robots' behavior in flexible robotic assembly lines. *Procedia manufacturing*, *28*, 121-126.

[20] – **Laaki, H., Miche, Y., & Tammi, K.** (2019). Prototyping a digital twin for real time remote control over mobile networks: Application of remote surgery. *Ieee Access*, *7*, 20325-20336.

[21] – **Tao, F., Zhang, H., Liu, A., & Nee, A. Y.** (2018). Digital twin in industry: State-of-the-art. *IEEE Transactions on industrial informatics*, *15*(4), 2405-2415

[22] – **Haag, S., & Anderl, R.** (2018). Digital twin–Proof of concept. *Manufacturing letters*, *15*, 64-66.

[23] – **Battle, R., & Benson, E.** (2008). Bridging the semantic Web and Web 2.0 with representational state transfer (REST). *Journal of Web Semantics*, *6*(1), 61-69.

APPENDIX

A1 Technical Drawings

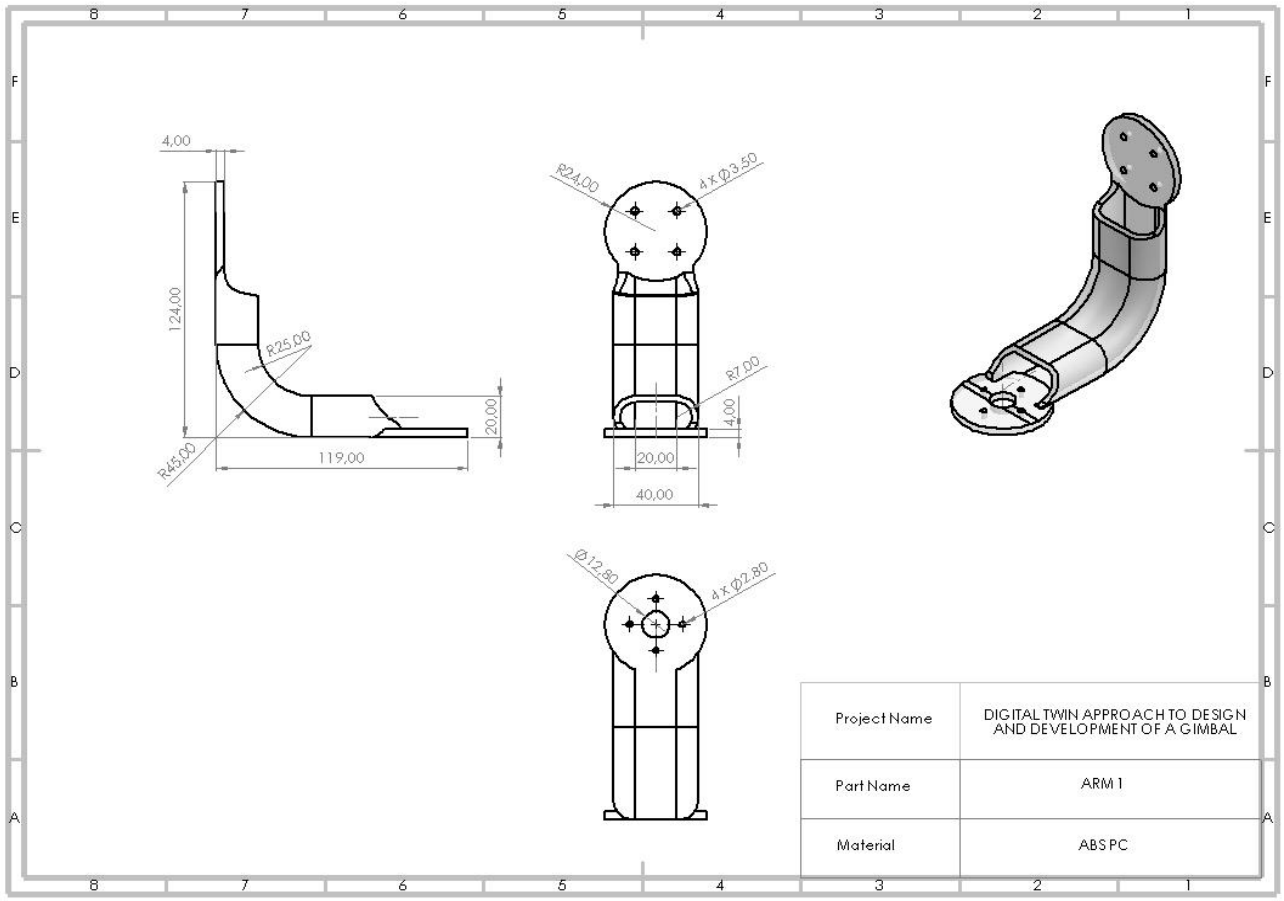


Figure A1.1 : Arm 1 Technical Drawing

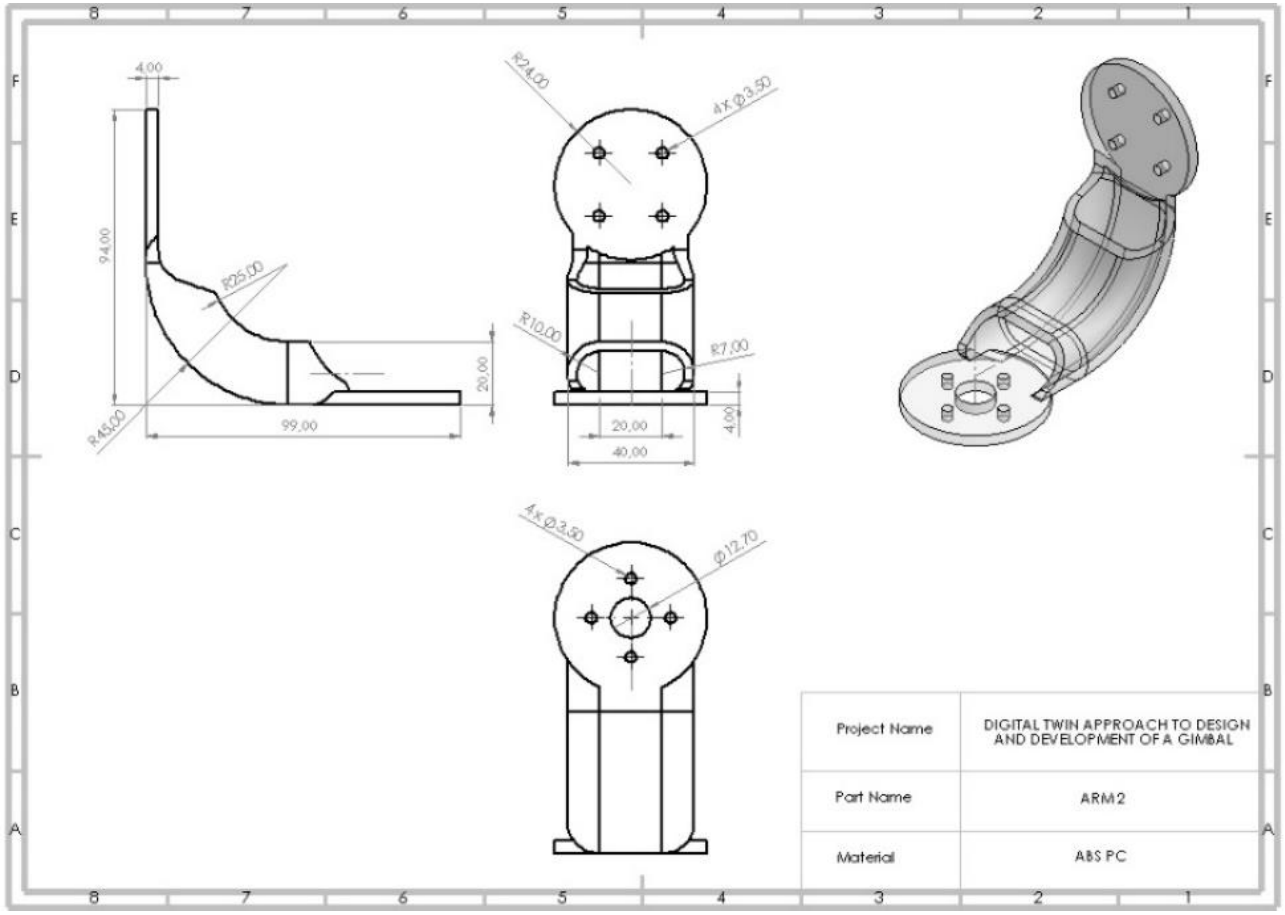


Figure A1.2 : Arm 2 Technical Drawing

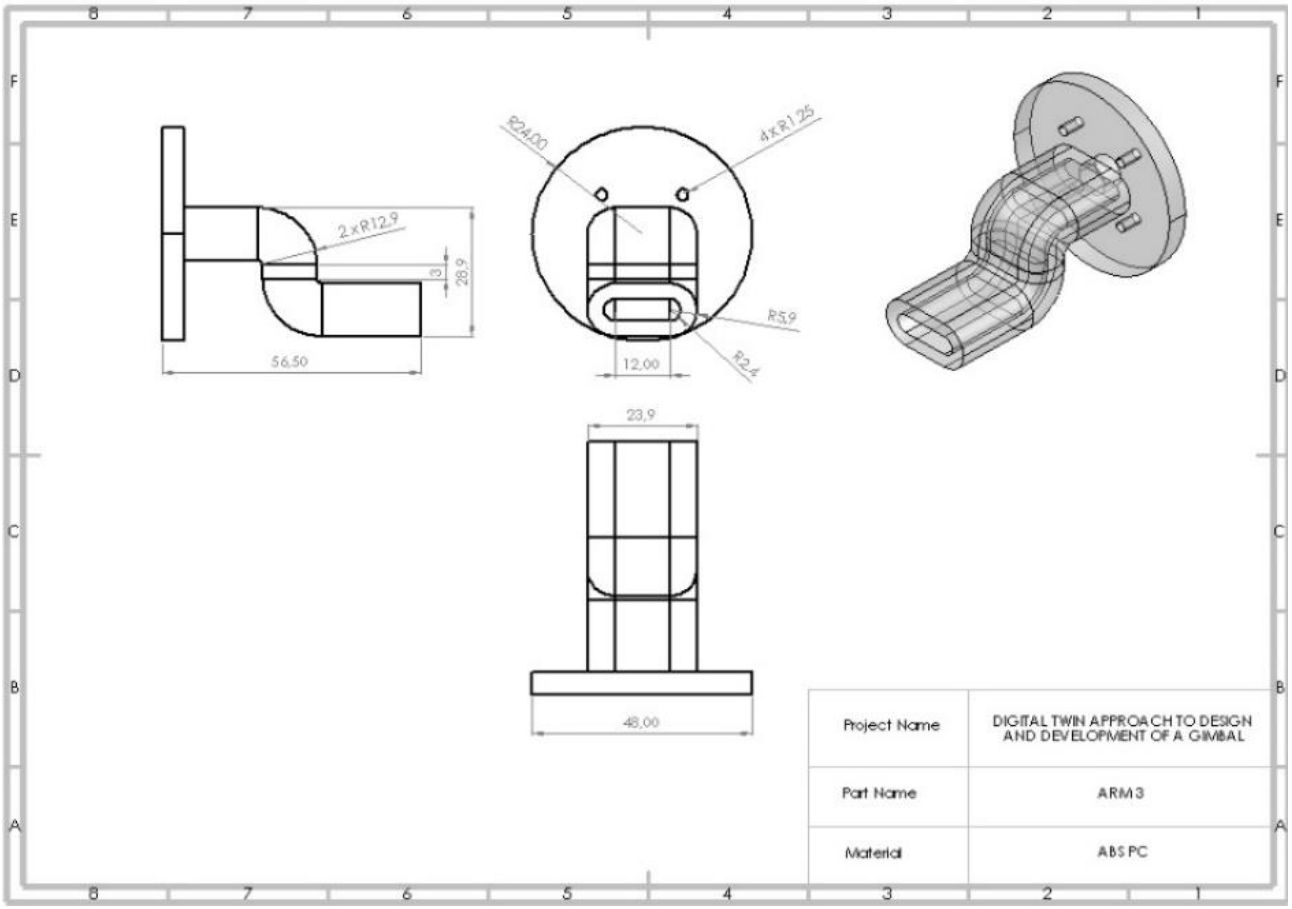


Figure A1.3 : Arm 3 Technical Drawing

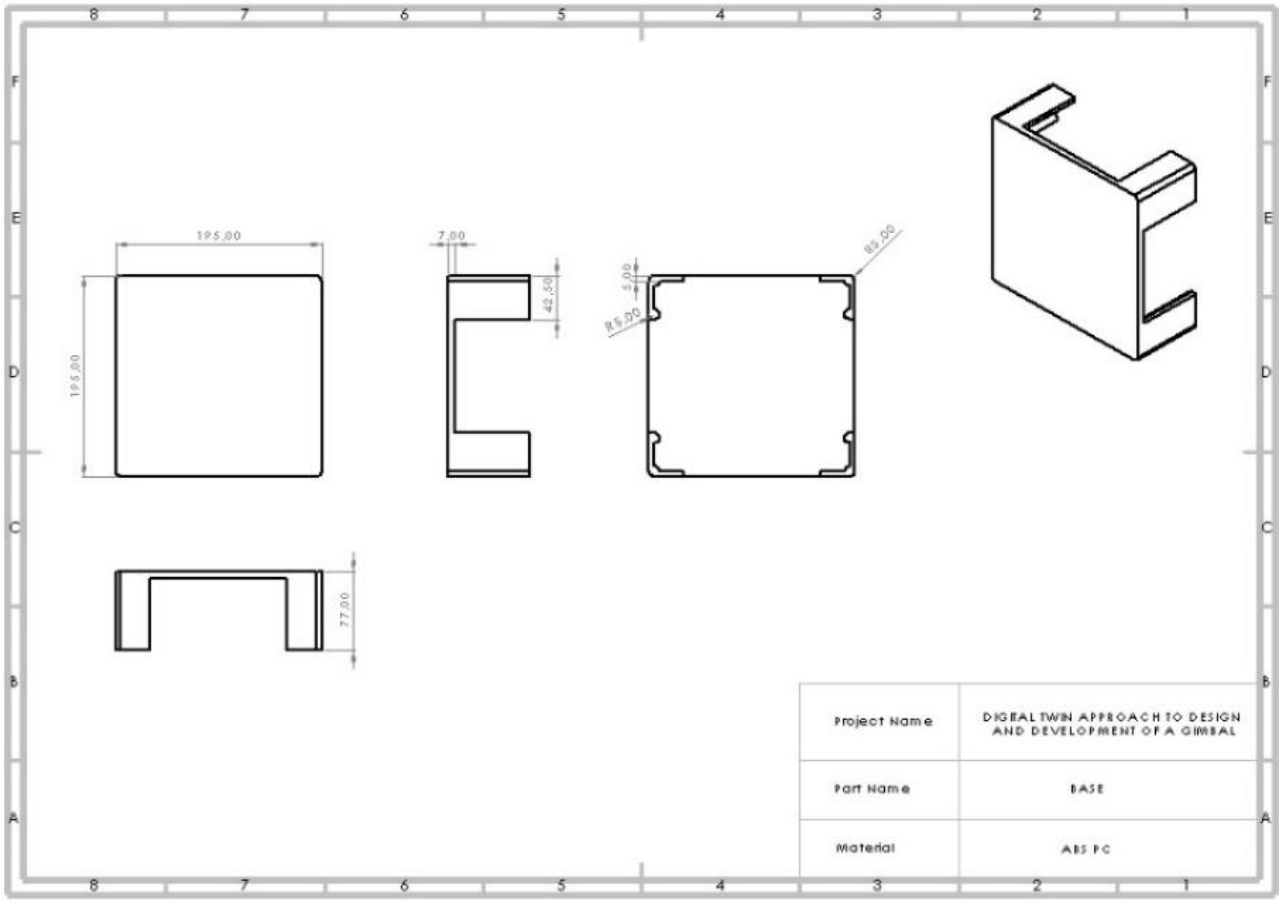


Figure A1.4 : Base Technical Drawing

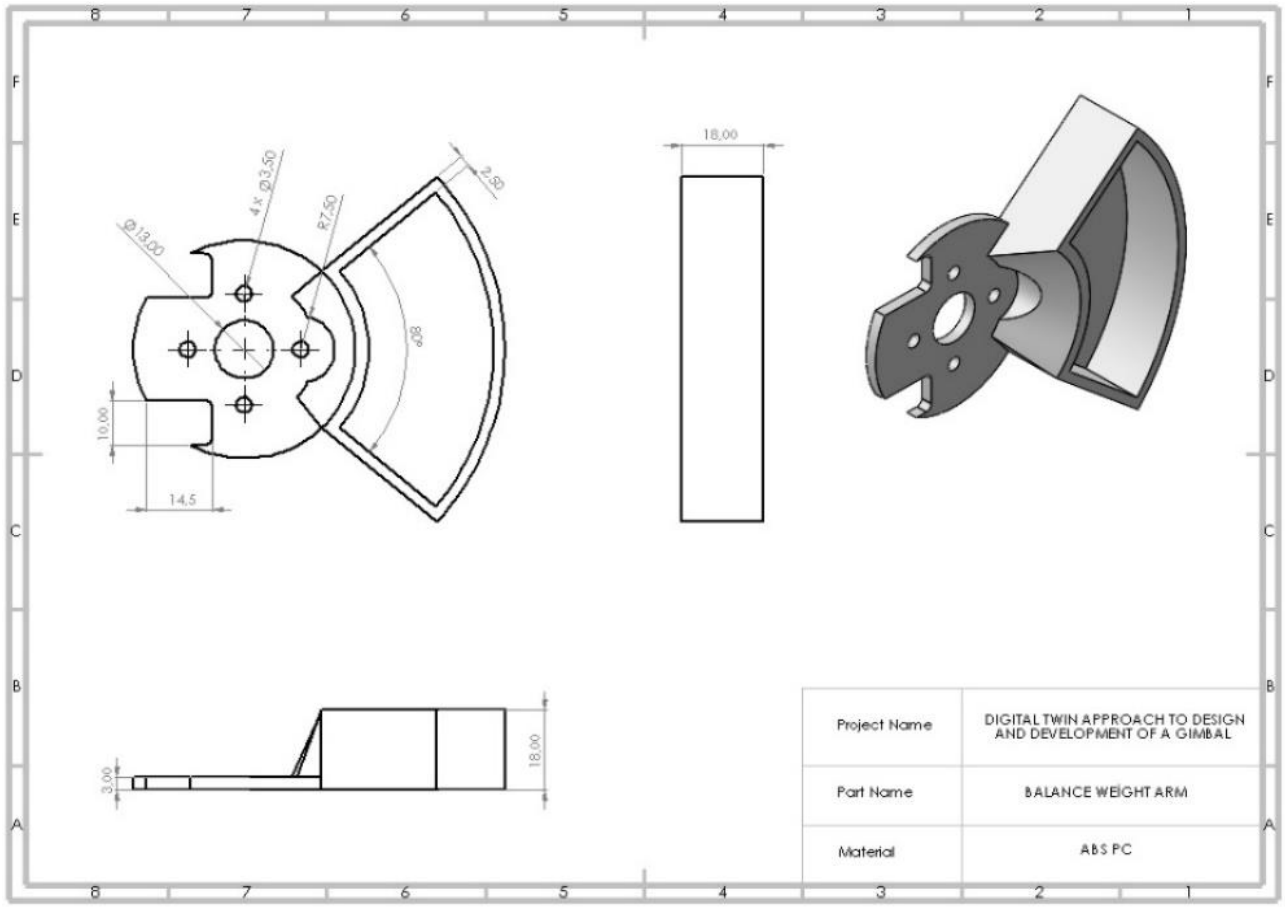


Figure A1.5 : Balance Weight Arm Technical Drawing

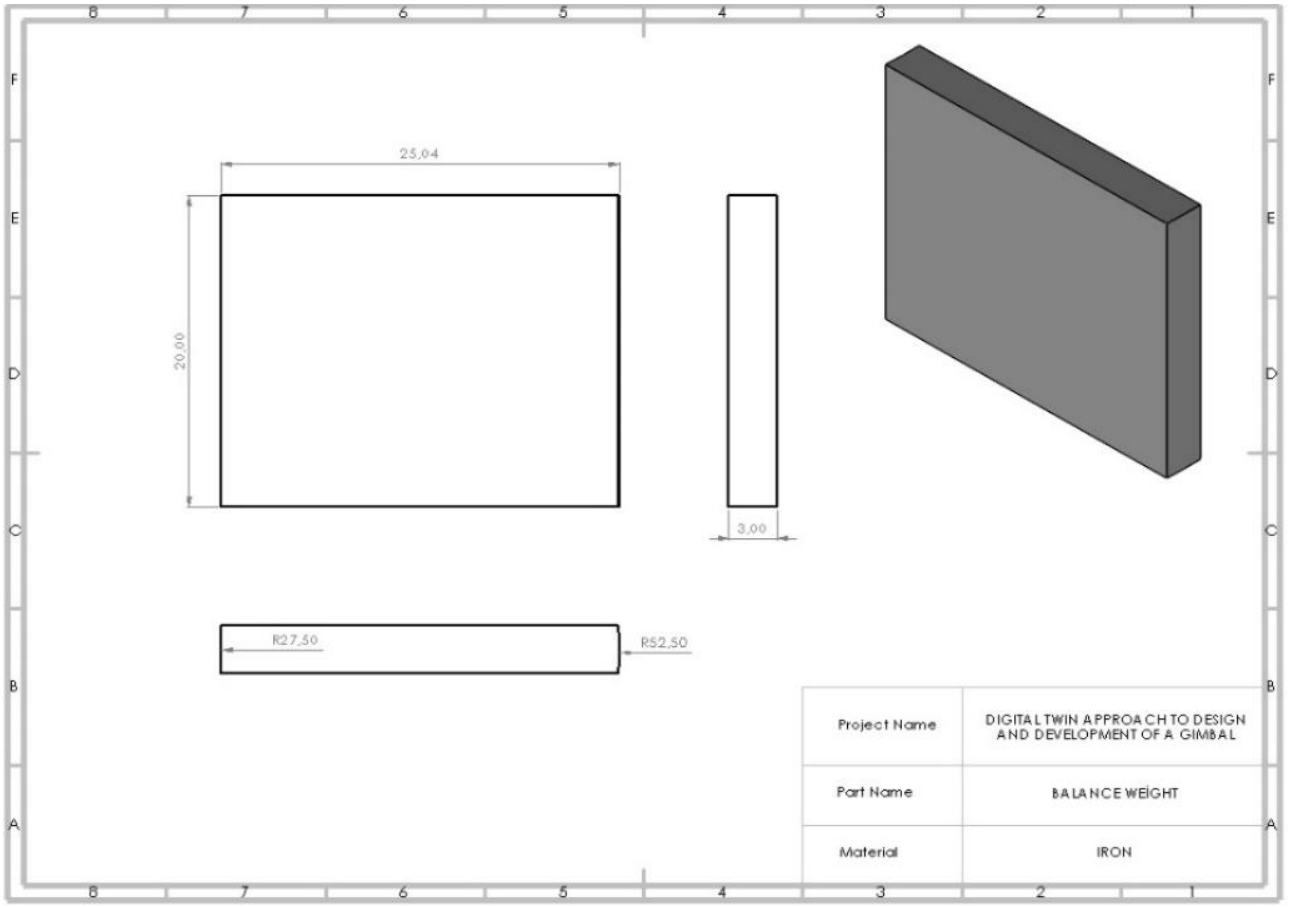
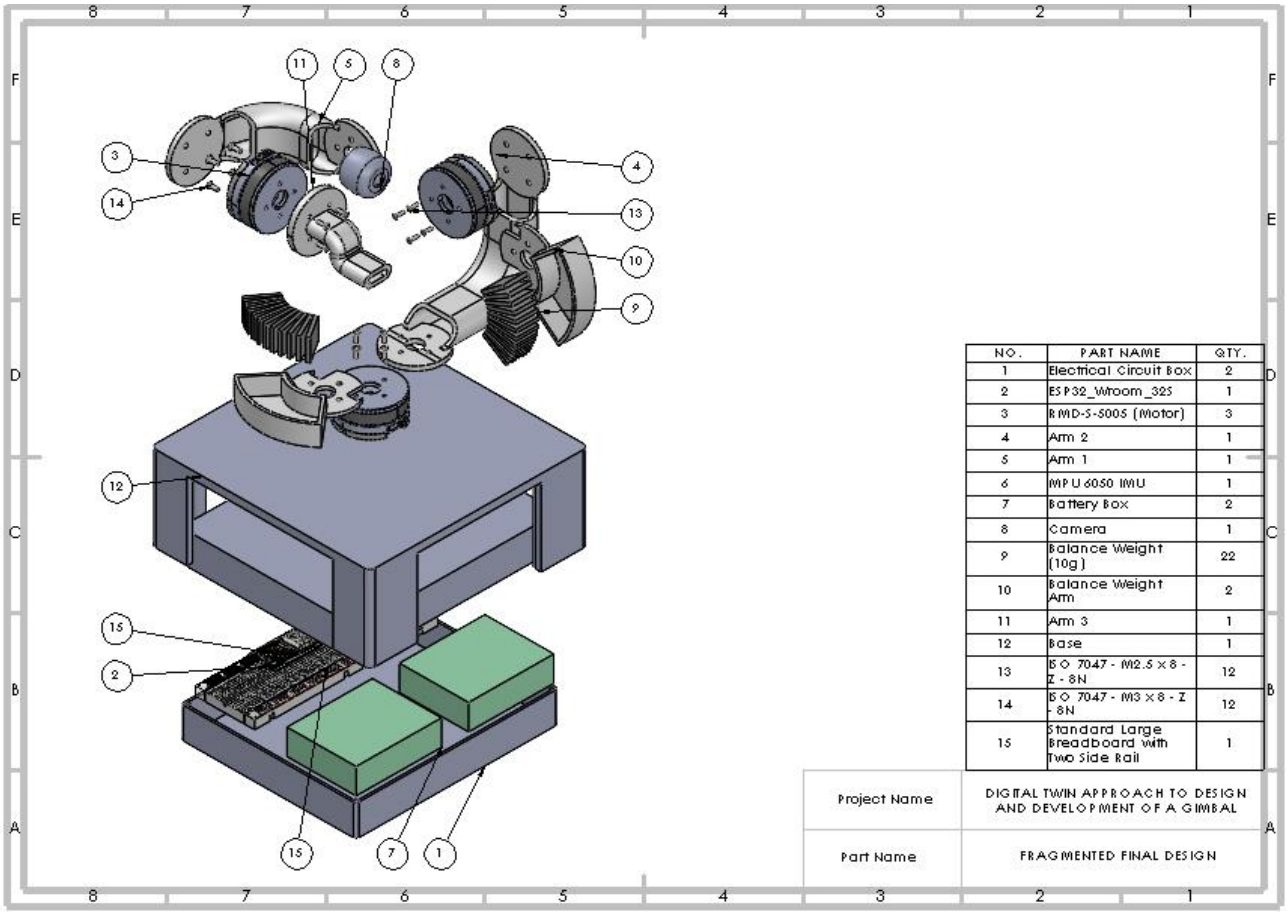


Figure A1.6 : Balance Weight Technical Drawing



NO.	PART NAME	QTY.
1	Electrical Circuit Box	2
2	ESP32_Wroom_32S	1
3	RMD-S-5005 (Motor)	3
4	Arm 2	1
5	Arm 1	1
6	MPU 6050 IMU	1
7	Battery Box	2
8	Camera	1
9	Balance Weight (10g)	22
10	Balance Weight Arm	2
11	Arm 3	1
12	Base	1
13	ISO 7047 - M2.5 x 8 - Z - 8N	12
14	ISO 7047 - M3 x 8 - Z - 8N	12
15	Standard Large Breadboard with Two Side Rail	1

Project Name	DIGITAL TWIN APPROACH TO DESIGN AND DEVELOPMENT OF A GIMBAL
Part Name	FRAGMENTED FINAL DESIGN

Figure A1.7 : Fragmented Final Design Technical Drawing

A2 Developed Computer Code

ARDUINO IDE:

```
#include <HardwareSerial.h>
#include "Math.h"
#include "I2Cdev.h"
#include "MPU6050_6Axis_MotionApps20.h"
#include "KickMath.h"
#include <WiFi.h>
#include <WebServer.h>
#include <ESPAsyncWebServer.h>
#include <ArduinoJson.h>

#include "BasicLinearAlgebra.h"
using namespace BLA;

#if I2CDEV_IMPLEMENTATION == I2CDEV_ARDUINO_WIRE
#include "Wire.h"
#endif

MPU6050 mpu;

AsyncWebServer server(80);

// Network Credentials
const char* ssid = "SUPERONLINE_WiFi_1CA5";
const char* password = "AXJ3NCCFLVYA";

void handleGetAll(AsyncWebServerRequest *request);
void handleSetTarget(AsyncWebServerRequest *request);
void handleSetPitchDownTargetTen(AsyncWebServerRequest *request);
void handleSetPitchUpTargetTen(AsyncWebServerRequest *request);
void handleSetYawRightTargetTen(AsyncWebServerRequest *request);
void handleSetYawLeftTargetTen(AsyncWebServerRequest *request);
void handleSetPitchDownTargetOne(AsyncWebServerRequest *request);
void handleSetPitchUpTargetOne(AsyncWebServerRequest *request);
void handleSetYawRightTargetOne(AsyncWebServerRequest *request);
void handleSetYawLeftTargetOne(AsyncWebServerRequest *request);
void handleSetPitchDownTargetFive(AsyncWebServerRequest *request);
void handleSetPitchUpTargetFive(AsyncWebServerRequest *request);
void handleSetYawRightTargetFive(AsyncWebServerRequest *request);
void handleSetYawLeftTargetFive(AsyncWebServerRequest *request);

float imuYaw;
float imuPitch;
float imuRoll;
float dtPitchMotor;
float dtYawMotor;
float dtRollMotor;
```

```
float setedTargetPitch = 0;
float setedTargetYaw = 0;
float yawMotorDeg;
float pitchMotorDeg;
float rollMotorDeg;

float palpha1_Desired = 220.00; // BLDC Motor 1 (Yaw Axis)(Z) angular value
Previous
float palpha2_Desired = 215.00; // BLDC Motor 2 (Pitch Axis)(Z) angular value
Previous
float palpha3_Desired = 90.00; // BLDC Motor 3 (Roll Axis)(Z) angular value
Previous
String yawMotor = "yaw";
String pitchMotor = "pitch";
String rollMotor = "roll";
// MPU control/status vars
bool dmpReady = false;  // set true if DMP init was successful
uint8_t devStatus;      // return status after each device operation (0 = success, !0 =
error)
uint16_t packetSize;    // expected DMP packet size (default is 42 bytes)
uint16_t fifoCount;     // count of all bytes currently in FIFO
uint8_t fifoBuffer[64]; // FIFO storage buffer

// orientation/motion vars
Quaternion q;           // [w, x, y, z]        quaternion container
VectorInt16 aa;         // [x, y, z]           accel sensor measurements
VectorInt16 gy;         // [x, y, z]           gyro sensor measurements
VectorInt16 aaReal;     // [x, y, z]           gravity-free accel sensor measurements
VectorInt16 aaWorld;    // [x, y, z]           world-frame accel sensor measurements
VectorFloat gravity;    // [x, y, z]           gravity vector
float euler[3];         // [psi, theta, phi]    Euler angle container
float ypr[3];           // [yaw, pitch, roll]   yaw/pitch/roll container and gravity vector

//GIMBAL VARIABLES************************
// Define variables for General
float psi;    // Yaw (Z) Axis angular value
float theta;  // Pitch Axis (Y) angular value
float gama;  // Roll Axis (X) angular value
float alpha1_Desired; // BLDC Motor 1 (Yaw Axis)(Z) angular value
float alpha2_Desired; // BLDC Motor 2 (Pitch Axis)(Y) angular value
float alpha3_Desired; // BLDC Motor 3 (Roll Axis)(X) angular value

// Define variables for INVERSE KINEMATICS
float targetYaw;   //Target yaw axis
float targetPitch; //Target pitch axis

BLA::Matrix<3, 3> targetOrientation;  //Target orientation matrix
BLA::Matrix<3, 3> targetOrientation_Base; //Target orientation wrt Base

BLA::Matrix<3, 3> baseYaw;    //Base Yaw Axis Rotation Matrix
```

```
BLA::Matrix<3, 3> basePitch;  //Base Pitch Axis Rotation Matrix
BLA::Matrix<3, 3> baseRoll;   //Base Roll Axis Rotation Matrix

BLA::Matrix<3, 3> baseYaw_t;    //Base Yaw Axis transpose Rotation Matrix
BLA::Matrix<3, 3> basePitch_t;  //Base Pitch Axis transpose Rotation Matrix
BLA::Matrix<3, 3> baseRoll_t;   //Base Roll Axis transpose Rotation Matrix

// ROTATION MATRIX GENERATION FUNCTIONS//////////////////////////
// Z-Axis Rotation Matrix
BLA::Matrix<3, 3> rotate_Z_so3(double Qz) {
  BLA::Matrix<3, 3> Rz;
  Rz = {cos(Qz), -sin(Qz), 0,
        sin(Qz),  cos(Qz), 0,
        0,        0, 1
       };
  return Rz;
}
// Y-Axis Rotation Matrix
BLA::Matrix<3, 3> rotate_Y_so3(double Qy) {
  BLA::Matrix<3, 3> Ry;
  Ry = { cos(Qy), 0, sin(Qy),
         0, 1,       0,
         -sin(Qy), 0, cos(Qy)
       };
  return Ry;
}
// X-Axis Rotation Matrix
BLA::Matrix<3, 3> rotate_X_so3(double Qx) {
  BLA::Matrix<3, 3> Rx;
  Rx = {1, 0, 0,
        0, cos(Qx), -sin(Qx),
        0, sin(Qx), cos(Qx)
       };
  return Rx;
}

////////////////////////////////////////////////////////////////////////////////

// angle değeri 90 ise 90.00 şeklinde giriniz
void motorSend(float angle, String axis, float currentAngle) {
  uint8_t message[10];
  message[0] = 0x3E;
  message[1] = 0xA5;
  if (axis == pitchMotor) {
    message[2] = 0x01;
  }
  else if (axis == yawMotor) {
    message[2] = 0x03;
  }
  else if (axis == rollMotor) {
```

```
  message[2] = 0x08;
}
message[3] = 0x04;

message[4] = 0;
for (int i = 0; i < 4; i++) {
  message[4] += message[i];
}

// 0x00 -> clockwise
// 0x01 -> Counterclockwise
if (angle < 180 && (currentAngle < 180)) {
  if ((angle - currentAngle) > 0) {
    message[5] = 0x00;
  }
  else {
    message[5] = 0x01;
  }
}
else if (angle > 180 && (currentAngle > 180)) {
  if ((angle - currentAngle) > 0) {
    message[5] = 0x00;
  }
  else {
    message[5] = 0x01;
  }
}
else if ((angle > 180 && (currentAngle < 180)) && (angle > 270)) {
  if ((angle - currentAngle - 360.00) > 0) {
    message[5] = 0x00;
  }
  else {
    message[5] = 0x01;
  }
}
else if ((angle > 180 && (currentAngle < 180)) && (angle < 270)) {
  if ((angle - currentAngle) > 0) {
    message[5] = 0x00;
  }
  else {
    message[5] = 0x01;
  }
}
else if ((angle < 180 && (currentAngle > 180)) && (angle < 90)) {
  if ((angle - currentAngle + 360.00) > 0) {
    message[5] = 0x00;
  }
  else {
    message[5] = 0x01;
  }
```

```
  }
  else if ((angle < 180 && (currentAngle > 180)) && (angle > 90)) {
    if ((angle - currentAngle) > 0) {
      message[5] = 0x00;
    }
    else {
      message[5] = 0x01;
    }
  }

  String angleHexa = decimalToHexa(angle);
  byte byteArray[2];

  for (int i = 0; i < angleHexa.length(); i += 2) {
    byteArray[i / 2] = strtoul(angleHexa.substring(i, i + 2).c_str(), NULL, 16);
  }

  message[6] = byteArray[0];
  message[7] = byteArray[1];

  message[8] = 0x00;

  message[9] = 0;
  for (int j = 5; j < 9; j++) {
    message[9] += message[j];
  }

  Serial.write(message, sizeof(message));
}
String decimalToHexa(float decimalnum) {
  decimalnum = decimalnum / 0.01;
  long quotient, remainder;
  int i, j = 0;
  char hexadecimalnum[100];

  quotient = decimalnum;
  while (quotient != 0)
  {
    remainder = quotient % 16;
    if (remainder < 10)
      hexadecimalnum[j++] = 48 + remainder;
    else
      hexadecimalnum[j++] = 55 + remainder;
    quotient = quotient / 16;
  }
  String hexa = String(hexadecimalnum[1]) + String(hexadecimalnum[0]) +
String(hexadecimalnum[3]) + String(hexadecimalnum[2]);
  return hexa;
}
```

```
void initWiFi() {
  WiFi.mode(WIFI_STA);
  WiFi.begin(ssid, password);
  Serial.print("Connecting to WiFi ..");
  while (WiFi.status() != WL_CONNECTED) {
    Serial.print('.');
    delay(1000);
  }
  Serial.println("");
  Serial.println("WiFi connected.");
  Serial.println("IP address: ");
  Serial.println(WiFi.localIP());
}

void setup() {

  // join I2C bus (I2Cdev library doesn't do this automatically)
#if I2CDEV_IMPLEMENTATION == I2CDEV_ARDUINO_WIRE
  Wire.begin();
  Wire.setClock(400000); // 400kHz I2C clock. Comment this line if having
compilation difficulties
#elif I2CDEV_IMPLEMENTATION == I2CDEV_BUILTIN_FASTWIRE
  Fastwire::setup(400, true);
#endif

  Serial.begin(115200);

  initWiFi();
  mpu.initialize();

  while (Serial.available() && Serial.read()); // empty buffer
  //Commented for
  //while (!Serial.available());                 // wait for data
  while (Serial.available() && Serial.read()); // empty buffer again

  devStatus = mpu.dmpInitialize();

  // supply your own gyro offsets here, scaled for min sensitivity
  mpu.setXAccelOffset(26);
  mpu.setYAccelOffset(1156);
  mpu.setZAccelOffset(740);
  mpu.setXGyroOffset(658);
  mpu.setYGyroOffset(-11);
  mpu.setZGyroOffset(-9);
  //****************************

  if (devStatus == 0) {

    mpu.setDMPEnabled(true);
```

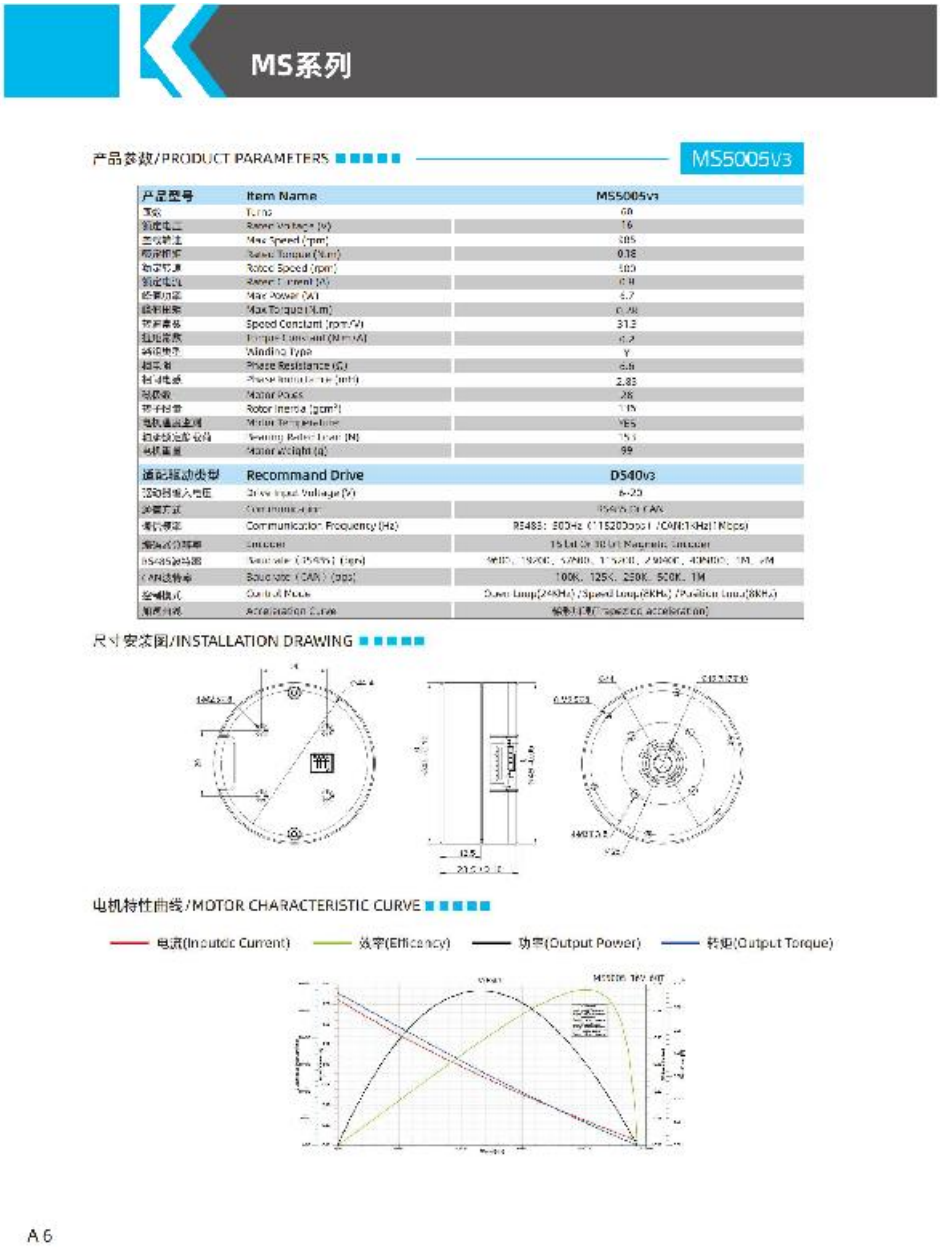
```
    dmpReady = true;

    packetSize = mpu.dmpGetFIFOPacketSize();
  }

  server.on("/getAll", HTTP_GET, handleGetAll);

  server.on("/setPitchUpTargetTen", HTTP_GET, handleSetPitchUpTargetTen);
  server.on("/setPitchDownTargetTen", HTTP_GET,
handleSetPitchDownTargetTen);
  server.on("/setYawRightTargetTen", HTTP_GET, handleSetYawRightTargetTen);
  server.on("/setYawLeftTargetTen", HTTP_GET, handleSetYawLeftTargetTen);

  server.on("/setPitchUpTargetOne", HTTP_GET, handleSetPitchUpTargetOne);
  server.on("/setPitchDownTargetOne", HTTP_GET,
handleSetPitchDownTargetOne);
  server.on("/setYawRightTargetOne", HTTP_GET, handleSetYawRightTargetOne);
  server.on("/setYawLeftTargetOne", HTTP_GET, handleSetYawLeftTargetOne);

  server.on("/setPitchUpTargetFive", HTTP_GET, handleSetPitchUpTargetFive);
  server.on("/setPitchDownTargetFive", HTTP_GET,
handleSetPitchDownTargetFive);
  server.on("/setYawRightTargetFive", HTTP_GET, handleSetYawRightTargetFive);
  server.on("/setYawLeftTargetFive", HTTP_GET, handleSetYawLeftTargetFive);

  DefaultHeaders::Instance().addHeader("Access-Control-Allow-Origin", "*");
  server.begin();

  delay(1000);
  motorSend(220.00, yawMotor, 359.99);
  delay(1000);
  motorSend(215.00, pitchMotor, 359.99);
  delay(1000);
  motorSend(0.00, rollMotor, 90.00);
  delay(5000);
}

void loop() {

  // if programming failed, don't try to do anything
  if (!dmpReady) return;
  // read a packet from FIFO
  if (mpu.dmpGetCurrentFIFOPacket(fifoBuffer)) { // Get the Latest packet

    // display Euler angles in degrees
    mpu.dmpGetQuaternion(&q, fifoBuffer);
    mpu.dmpGetGravity(&gravity, &q);
    mpu.dmpGetYawPitchRoll(ypr, &q, &gravity);

    // IMU to Gimbal connection**********************
```

```
psi = -ypr[0];
theta = ypr[2];
gama = ypr[1];

targetYaw = setedTargetYaw * M_PI / 180;
targetPitch = setedTargetPitch * M_PI / 180;

// Target Orientation Creation//////////////
targetOrientation = rotate_Z_so3(targetYaw) * rotate_Y_so3(targetPitch);

// Base Rotation Matrices and their transpose
baseYaw = rotate_Z_so3(psi);      //Base yaw(Z) rotation matrix
baseYaw_t = ~baseYaw;             //  its transpose

basePitch = rotate_Y_so3(theta);  //Base pitch(Y) rotation matrix
basePitch_t = ~basePitch;         //  its transpose

baseRoll = rotate_X_so3(gama);   //Base roll(X) rotation matrix
baseRoll_t = ~baseRoll;           //  its transpose

// Representing Target Orientation wrt Base
targetOrientation_Base = baseRoll_t*basePitch_t*baseYaw_t*targetOrientation;

// Inverse Kinematics For BLDC Motor angles

alpha1_Desired = atan2(targetOrientation_Base(1, 0), targetOrientation_Base(0,
0));
alpha2_Desired = -atan2(-targetOrientation_Base(2, 0),
sqrt(sq(targetOrientation_Base(0, 0)) + sq(targetOrientation_Base(1, 0))));
alpha3_Desired = atan2(targetOrientation_Base(2, 1), targetOrientation_Base(2,
2));

dtPitchMotor = alpha2_Desired;
dtYawMotor = alpha1_Desired;
dtRollMotor = alpha3_Desired;

alpha1_Desired = alpha1_Desired - (140 * M_PI / 180);
alpha2_Desired = alpha2_Desired - (145 * M_PI / 180);
alpha3_Desired = alpha3_Desired - (270 * M_PI / 180);

// Motor Control
yawMotorDeg = alpha1_Desired * 180 / M_PI;
pitchMotorDeg = alpha2_Desired * 180 / M_PI;
rollMotorDeg = alpha3_Desired * 180 / M_PI;

// For motor control negative values are corrected with 360 degree
if (yawMotorDeg < 0) {
  yawMotorDeg = yawMotorDeg + 360.00;
}
if (pitchMotorDeg < 0) {
```

```
    pitchMotorDeg = pitchMotorDeg + 360.00;
   }
   if (rollMotorDeg < 0) {
    rollMotorDeg = rollMotorDeg + 360.00;
   }

   motorSend(yawMotorDeg, yawMotor, palpha1_Desired);
   delay(100);
   motorSend(pitchMotorDeg, pitchMotor, palpha2_Desired);
   delay(100);
   motorSend(rollMotorDeg, rollMotor, palpha3_Desired);
   delay(100);

   // Previous angles
   palpha1_Desired = yawMotorDeg;
   palpha2_Desired = pitchMotorDeg;
   palpha3_Desired = rollMotorDeg;
   imuYaw = psi;
   imuPitch = theta;
   imuRoll = gama;

  }
}
void handleGetAll(AsyncWebServerRequest *request) {
 DynamicJsonDocument jsonDoc(256);
 JsonObject jsonData = jsonDoc.to<JsonObject>();

 jsonData["rollMotor"] = dtRollMotor;
 jsonData["yawMotor"] = dtYawMotor;
 jsonData["pitchMotor"] = dtPitchMotor;
 jsonData["imuYaw"] = imuYaw;
 jsonData["imuRoll"] = imuRoll;
 jsonData["imuPitch"] = imuPitch;
 jsonData["pitchTarget"] = targetPitch * 180 / M_PI;
 jsonData["yawTarget"] = targetYaw * 180 / M_PI;

 String jsonString;
 serializeJson(jsonData, jsonString);

 request->send(200, "application/json", jsonString);
}

void handleSetPitchUpTargetTen(AsyncWebServerRequest *request) {
 setedTargetPitch += 10;
 request->send(200, "text/plain", "value updated");
}

void handleSetPitchDownTargetTen(AsyncWebServerRequest *request) {
 setedTargetPitch -= 10;
 request->send(200, "text/plain", "value updated");
```

```
}

void handleSetYawRightTargetTen(AsyncWebServerRequest *request) {
  setedTargetYaw += 10;
  request->send(200, "text/plain", "value updated");
}

void handleSetYawLeftTargetTen(AsyncWebServerRequest *request) {
  setedTargetYaw -= 10;
  request->send(200, "text/plain", "value updated");
}

void handleSetPitchUpTargetOne(AsyncWebServerRequest *request) {
  setedTargetPitch += 1;
  request->send(200, "text/plain", "value updated");
}

void handleSetPitchDownTargetOne(AsyncWebServerRequest *request) {
  setedTargetPitch -= 1;
  request->send(200, "text/plain", "value updated");
}

void handleSetYawRightTargetOne(AsyncWebServerRequest *request) {
  setedTargetYaw += 1;
  request->send(200, "text/plain", "value updated");
}

void handleSetYawLeftTargetOne(AsyncWebServerRequest *request) {
  setedTargetYaw -= 1;
  request->send(200, "text/plain", "value updated");
}
void handleSetPitchUpTargetFive(AsyncWebServerRequest *request) {
  setedTargetPitch += 5;
  request->send(200, "text/plain", "value updated");
}

void handleSetPitchDownTargetFive(AsyncWebServerRequest *request) {
  setedTargetPitch -= 5;
  request->send(200, "text/plain", "value updated");
}

void handleSetYawRightTargetFive(AsyncWebServerRequest *request) {
  setedTargetYaw += 5;
  request->send(200, "text/plain", "value updated");
}

void handleSetYawLeftTargetFive(AsyncWebServerRequest *request) {
  setedTargetYaw -= 5;
  request->send(200, "text/plain", "value updated");
}
```

A3 Supplementary Materials



A 6

Figure A3.1 : Motor Product Parameters Sheet.